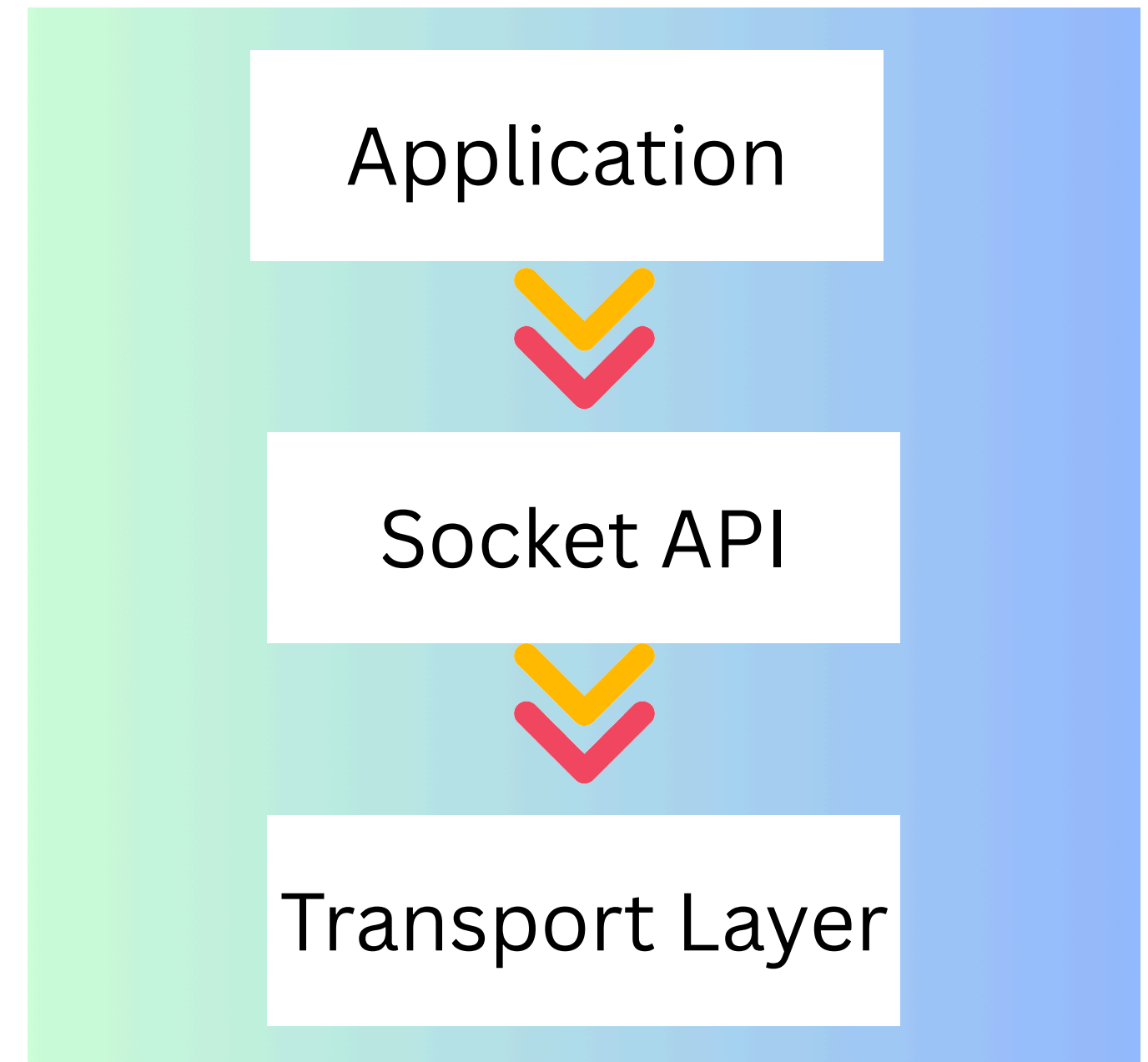


Socket API as an Interface b/w the Application and Transport Layers

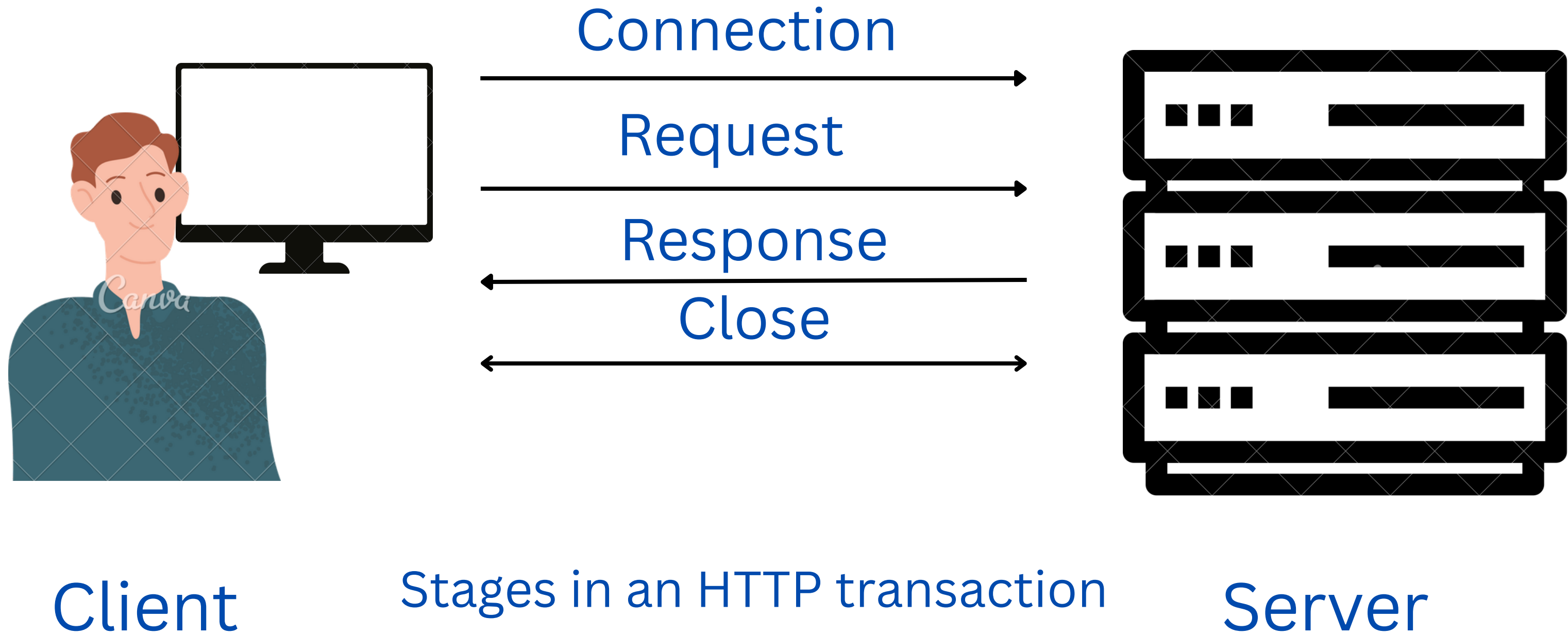
- The Socket API is a toolkit enabling apps to communicate with the OSI model's transport layer.
- It serves as a bridge for apps to connect with other systems using protocols like TCP and UDP.



Application Layer Protocols

- HTTP - Hypertext Transfer Protocol
- HTTPS - Hypertext Transfer Protocol Secure
- HTTP/2

HTTP



HTTP

- HTTP is a stateless protocol operating over TCP.

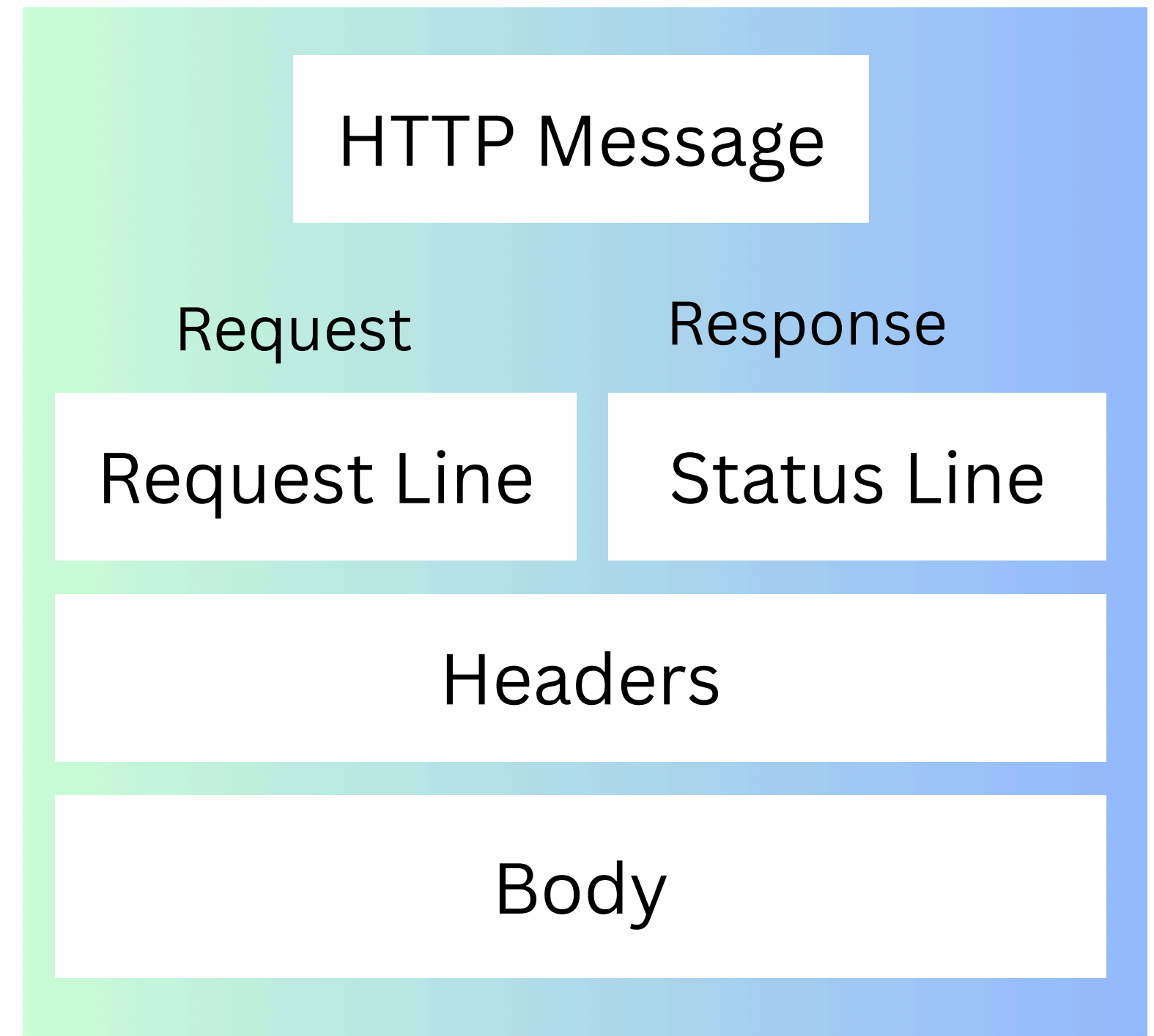
Stateless Nature of HTTP:

- Each HTTP request is independent, treated as a new request with no memory of prior interactions.
- The server does not retain information about previous client requests or sessions.

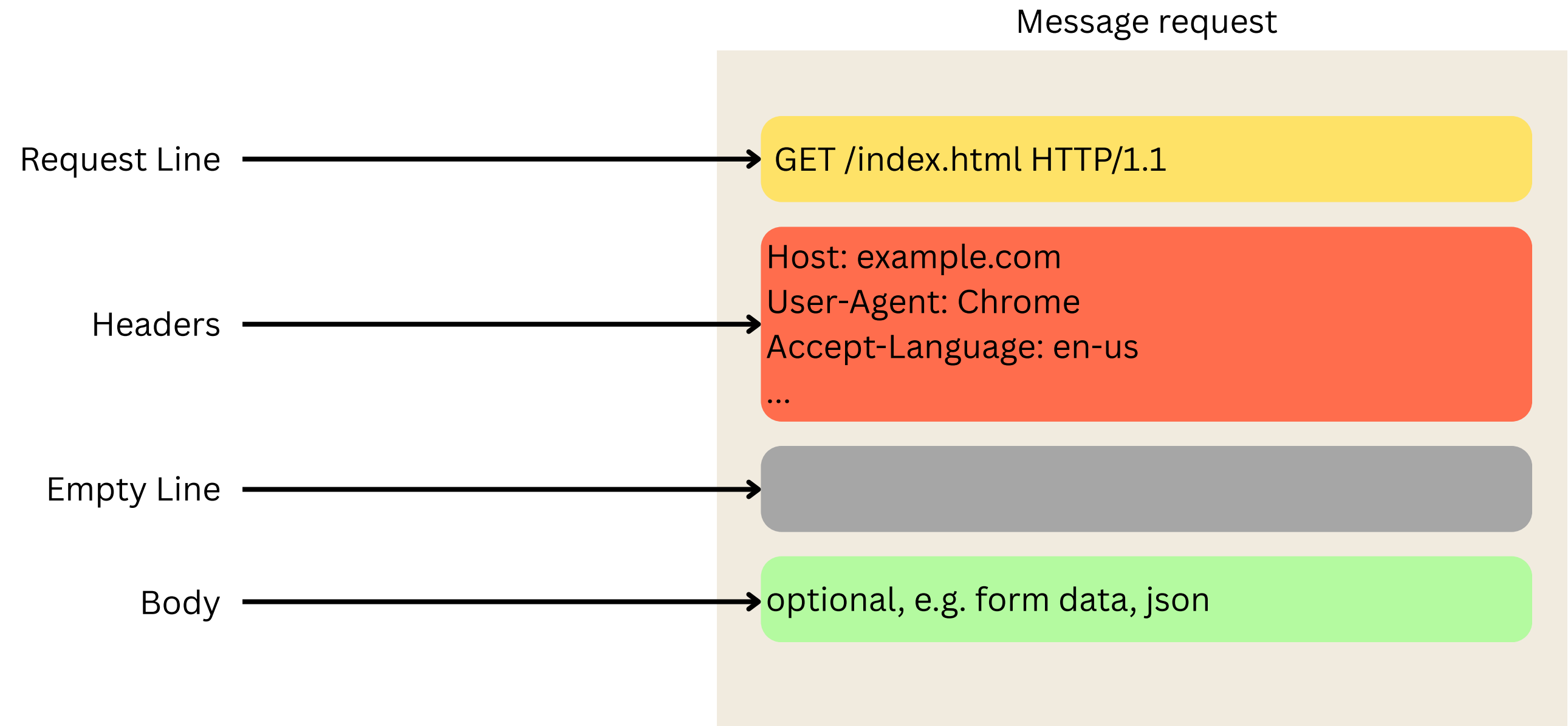


Breakdown of HTTP Messages

- HTTP messages consist of **requests** (sent from the client to the server) and **responses** (sent from the server to the client).
- These messages are structured in a way that includes a request/response line, headers, and optionally, a body.



HTTP Request Message Breakdown:



HTTP Request Message Breakdown:

Request Line:

- The request line contains three parts:
 - **Method:** The type of operation the client wants the server to perform (e.g., GET, POST, PUT, DELETE).
 - **URL:** The resource or endpoint on the server that the client is requesting (e.g., /home, /api/data).
 - **HTTP Version:** The version of HTTP being used (e.g., HTTP/1.1 or HTTP/2).
 - **Example:**
 - GET /index.html HTTP/1.1

HTTP Request Message Breakdown:



Headers:

- Headers provide additional information about the request or about the client's capabilities.
- **Common headers include:**
 - **User-Agent:** Information about the client software (browser, app, etc.).
 - **Host:** The domain name of the server.
 - **Content-Type:** The type of data the client is sending (for POST or PUT requests).
 - **Accept:** The types of data the client can process (e.g., text/html, application/json).

HTTP Request Message Breakdown:

Body (optional):

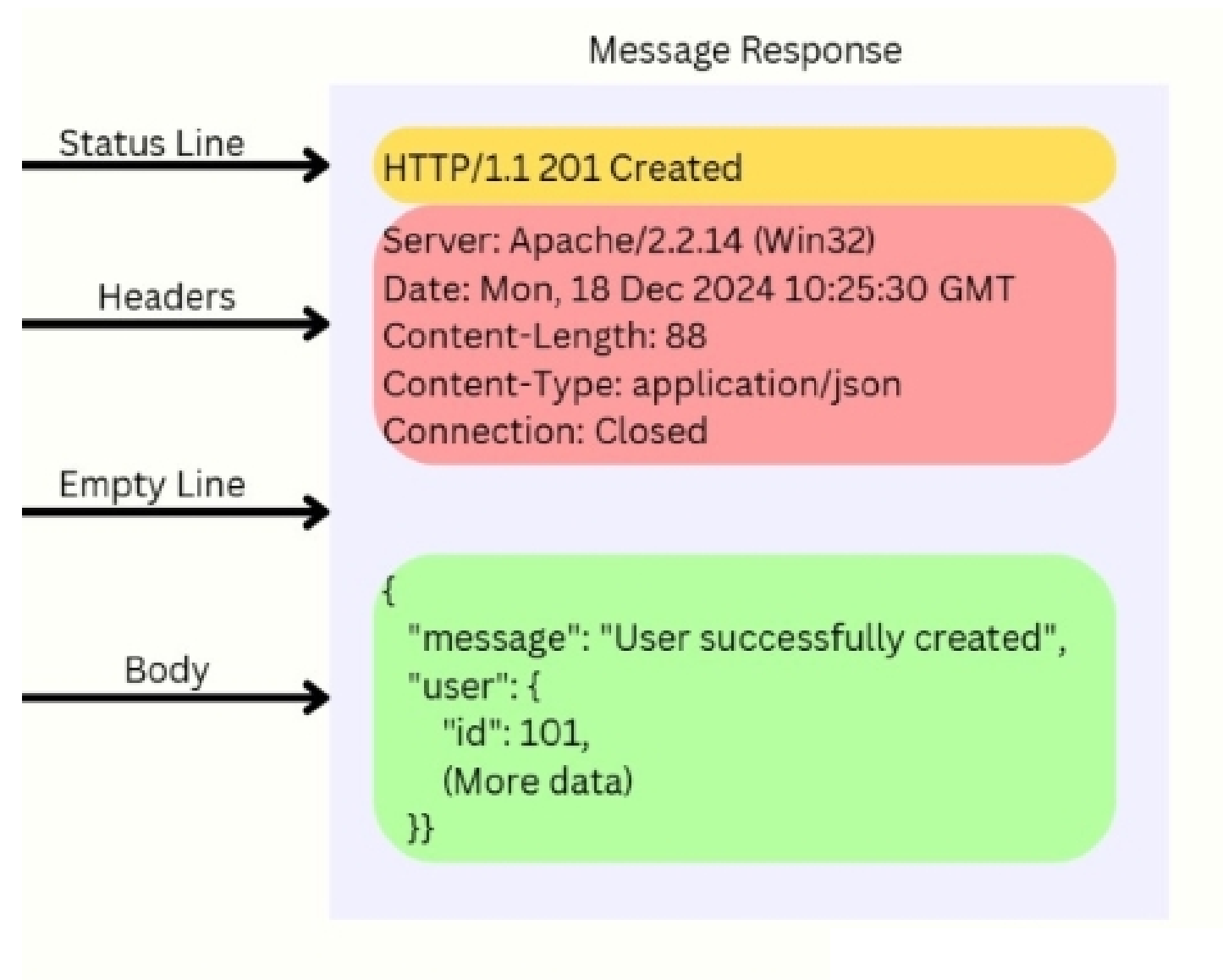
- The body contains the actual data being sent by the client (such as form data, JSON payload, etc.).
- This is only included for certain methods like POST, PUT, or PATCH.

Example (for POST request):

json

```
{ "username": "user1", "password": "password123" }
```

HTTP Response Message Breakdown:



HTTP Response Message

Breakdown:

Status Line:

- **The status line consists of:**
 - **HTTP Version:** The HTTP version used in the response.
 - **Status Code:** A 3-digit code indicating the result of the request (e.g., 200, 404, 500).
 - **Status Message:** A short description of the status code (e.g., OK, Not Found, Internal Server Error).

Example:

HTTP/1.1 200 OK

HTTP Response Message

Breakdown:

Headers:

- Response headers contain metadata about the response, similar to request headers.
- **Common headers include:**
 - **Content-Type:** The type of data being sent in the body (e.g., text/html, application/json).
 - **Content-Length:** The length of the body in bytes.
 - **Server:** Information about the server software.
 - **Date:** The date and time the response was sent.

HTTP Response Message

Breakdown:

Body (optional):

- The body of the response contains the actual data being returned by the server (e.g., HTML, JSON, an image, etc.).

Example:

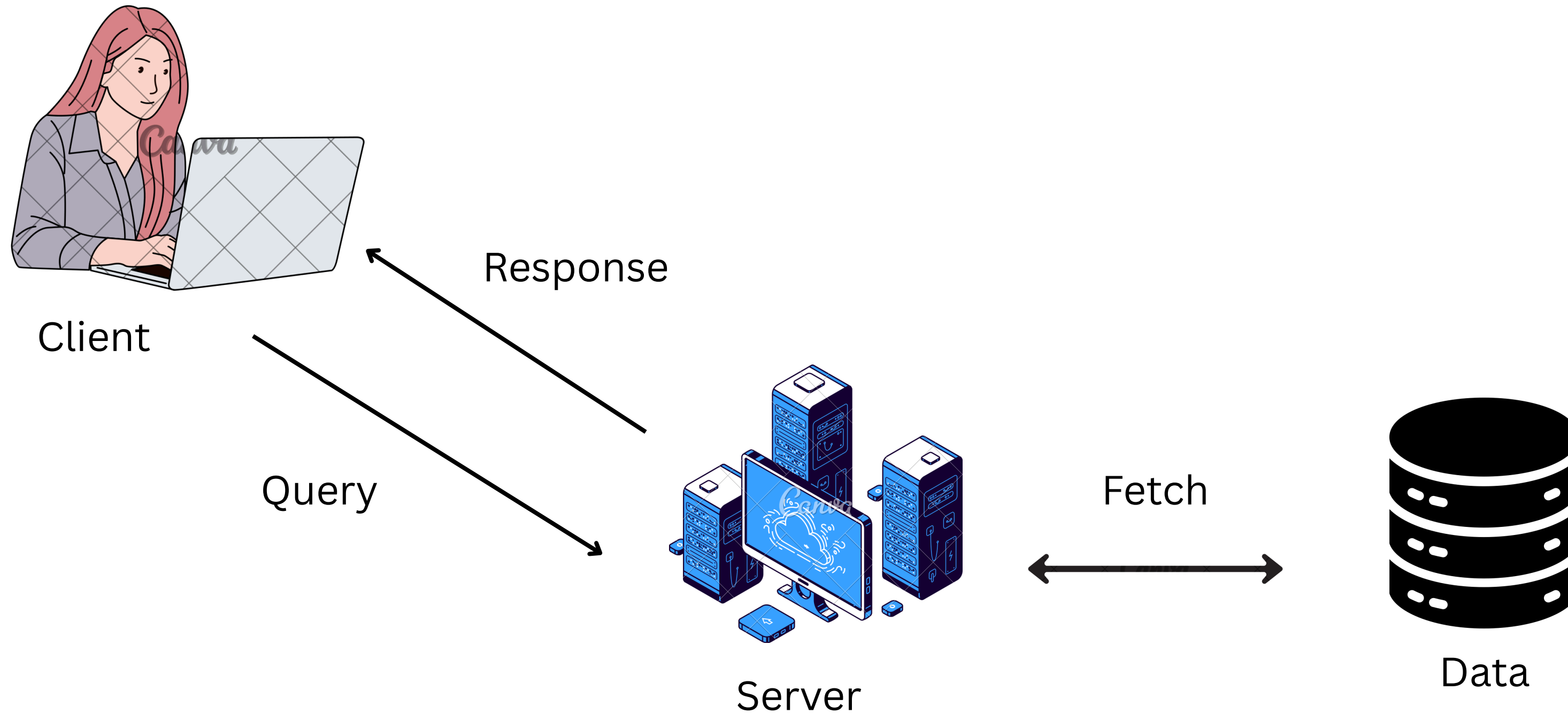
CSS

```
<html><body><h1>Welcome to Example.com</h1></body></html>
```

Demo: create DNS records in AWS Route 53

Let's discuss about APIs a bit

Client Server Paradigm



Jeff Bezos: API Mandate in 2002

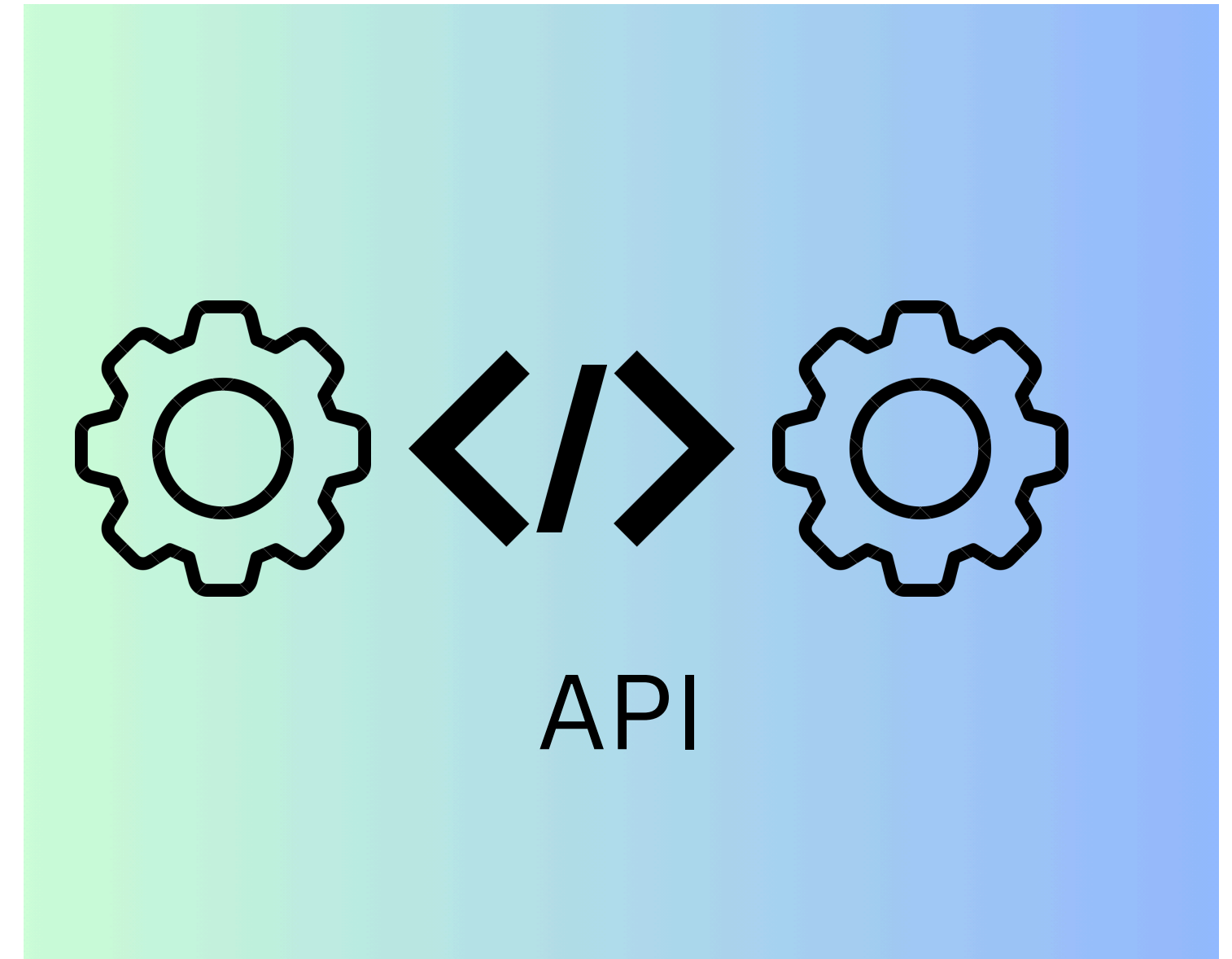
(paraphrased)

- All teams will henceforth expose their data and functionality through service interfaces.
- Teams must communicate with each other through these service interfaces.
- No other communication is allowed other than service interfaces over the network.
- It doesn't matter what technology they use.
- All service interfaces must be designed to be externalizable.
- Anyone who doesn't do this will be fired.



API

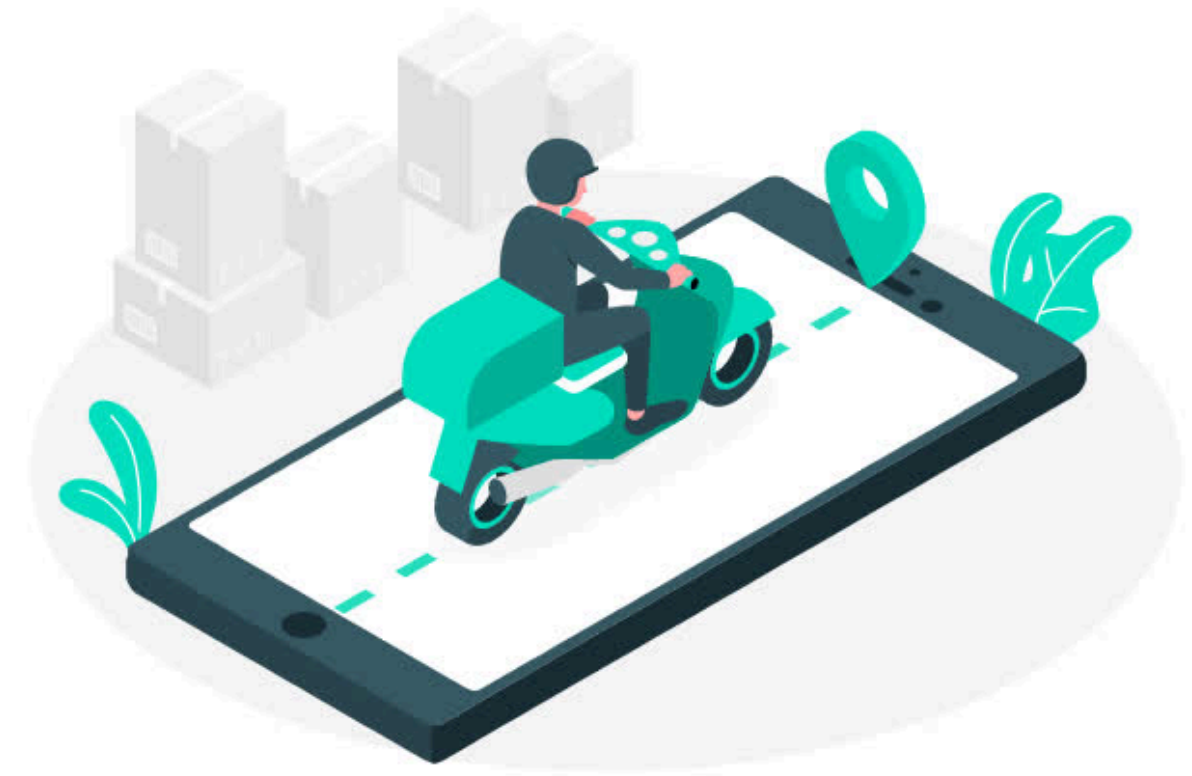
- An API (Application Programming Interface) is a set of protocols, routines, and tools that allow different software applications or systems to communicate with each other over a network.
- APIs define how software components should interact, providing a structured way for systems to send and receive data.



Google Maps API

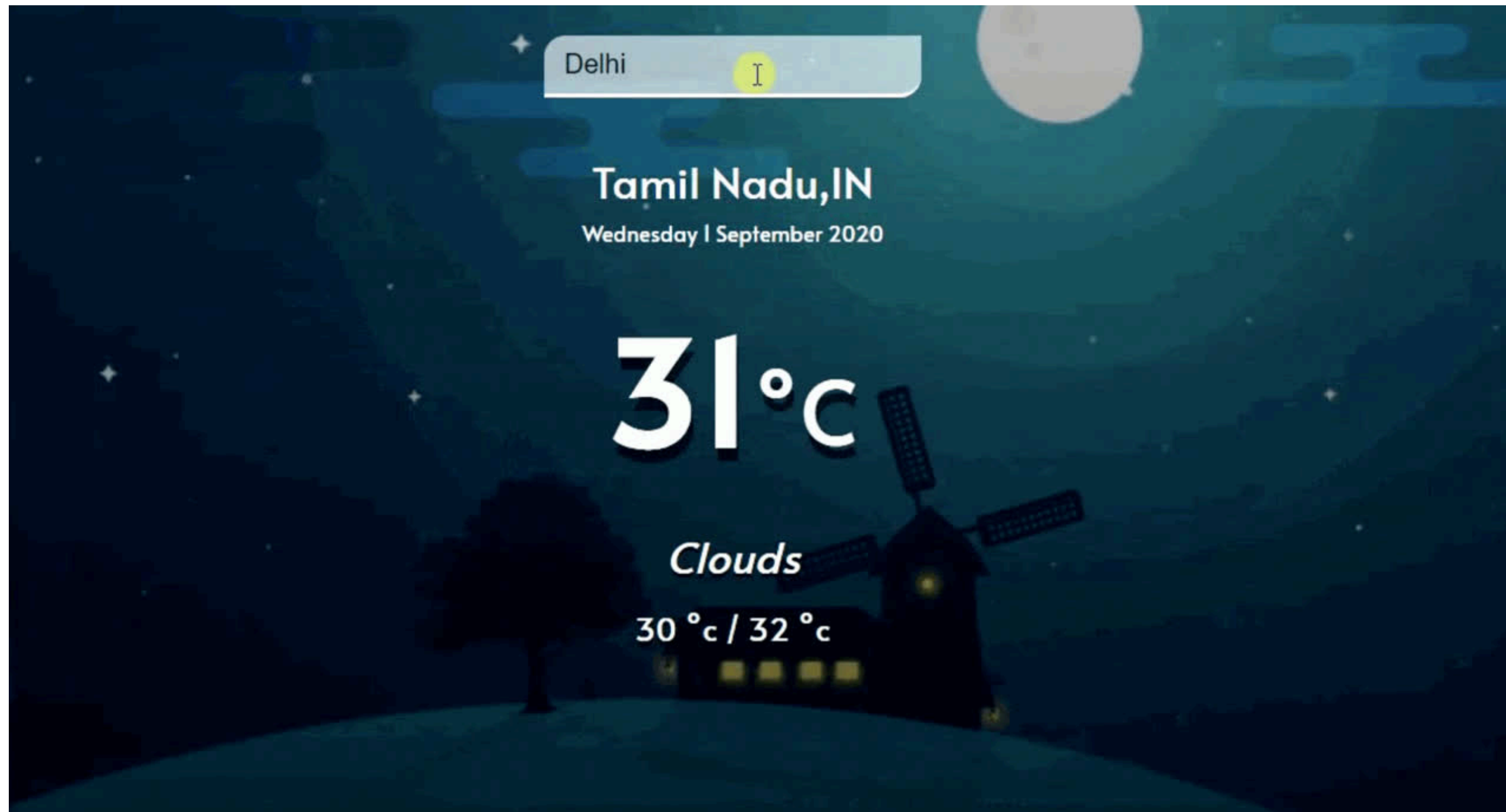


Location-Based Apps



Delivery Apps

Weather App



HTTP Response Message

Breakdown:

Body (optional):

- The body of the response contains the actual data being returned by the server (e.g., HTML, JSON, an image, etc.).

Example:

CSS

```
<html><body><h1>Welcome to Example.com</h1></body></html>
```

Substitution Cipher

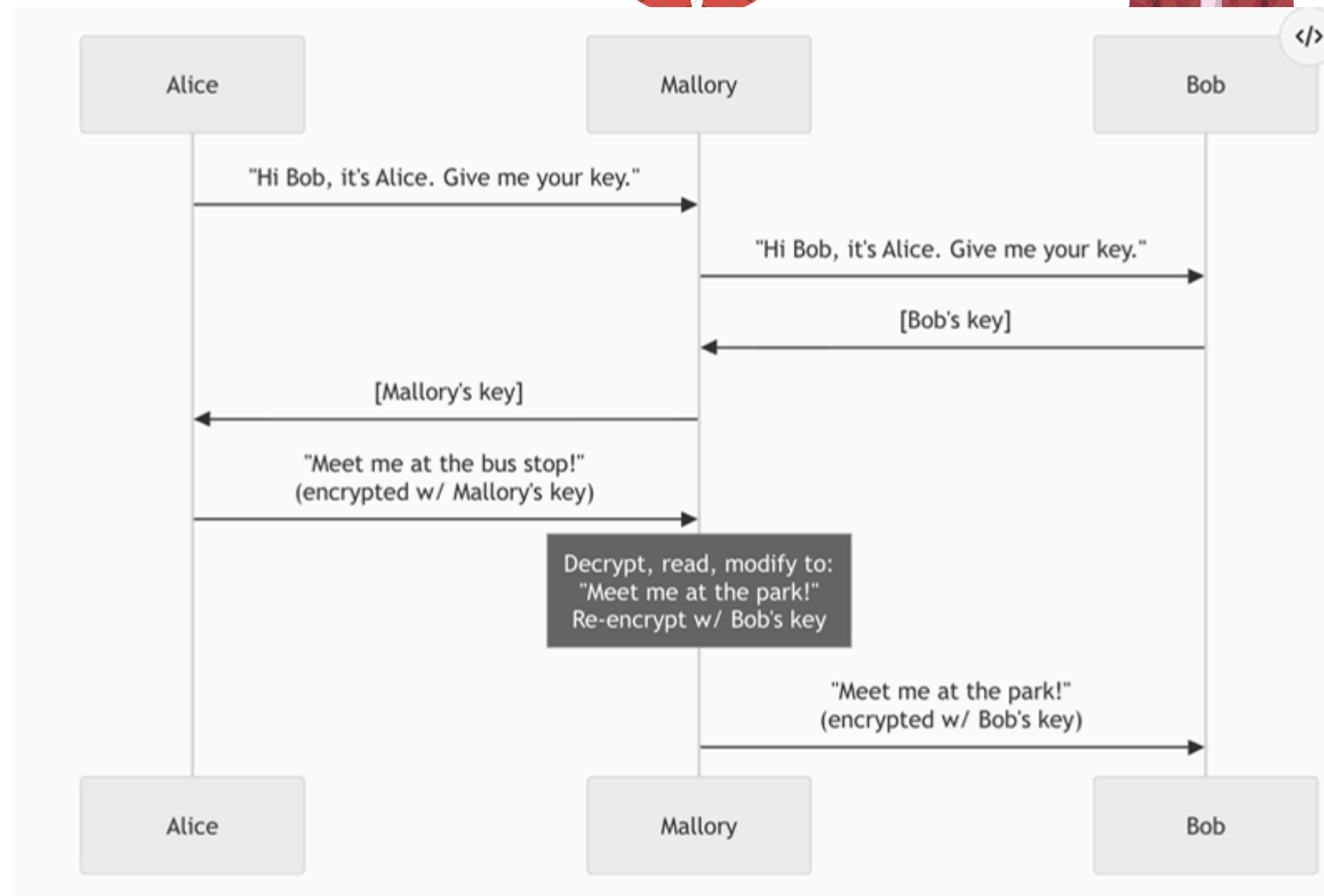
Find out what the concealed message says, the shift by number is not known, you just have the text

Gztkliv rsyz srrbz yrz dviv ufjk

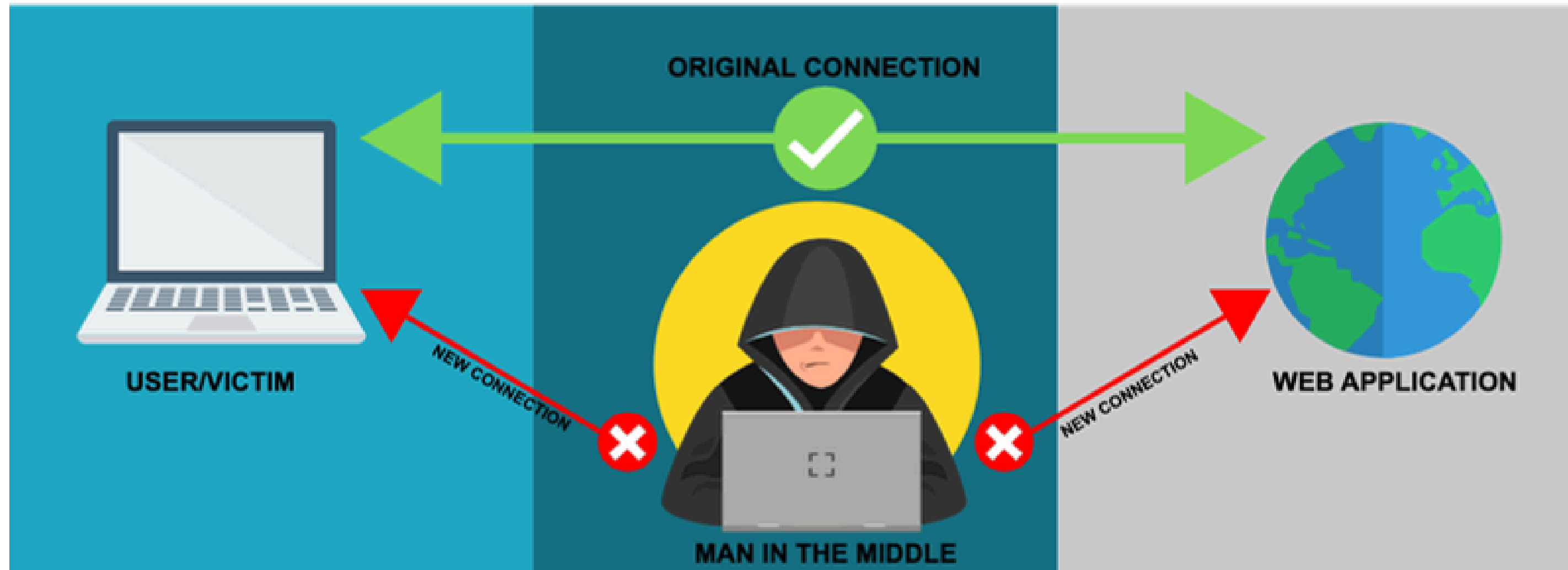
Do it without using LLMs, online tools are game! Simple python brute force script would be game as well!

Alice, Bob & Mallory from the world of cryptography

Alice & Bob & Mallory, since 1978

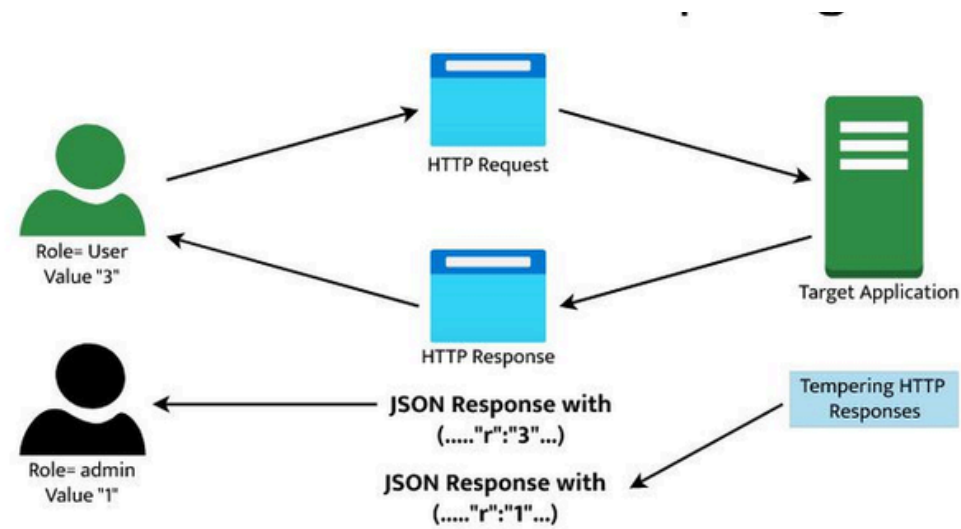


Man in the middle attack



Simply put, a MITM (Man-in-the-Middle) attack is the type of online attack where a hacker gets in between a user and the website they're visiting.

The Challenge: Communicating Secrets in Public



Tampering



Eavesdropping



Impersonation

What Mallory has to say...

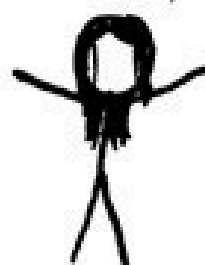
I'M SURE YOU'VE HEARD ALL ABOUT THIS SORDID AFFAIR IN THOSE GOSSIPY CRYPTOGRAPHIC PROTOCOL SPECS WITH THOSE BUSYBODIES SCHNEIER AND RIVEST, ALWAYS TAKING ALICE'S SIDE, ALWAYS LABELING ME THE ATTACKER.



YES, IT'S TRUE. I BROKE BOB'S PRIVATE KEY AND EXTRACTED THE TEXT OF HER MESSAGES. BUT DOES ANYONE REALIZE HOW MUCH IT HURT?



HE SAID IT WAS NOTHING, BUT EVERYTHING FROM THE PUBLIC-KEY AUTHENTICATED SIGNATURES ON THE FILES TO THE LIPSTICK HEART SMEARED ON THE DISK SCREAMED "ALICE."



I DIDN'T WANT TO BELIEVE. OF COURSE ON SOME LEVEL I REALIZED IT WAS A KNOWN-PLAINTEXT ATTACK. BUT I COULDN'T ADMIT IT UNTIL I SAW FOR MYSELF.



SO BEFORE YOU SO QUICKLY LABEL ME A THIRD PARTY TO THE COMMUNICATION, JUST REMEMBER: I LOVED HIM FIRST. WE HAD SOMETHING AND SHE TORE IT AWAY. SHE'S THE ATTACKER, NOT ME.



NOT ~~EVE~~.
Mallory.

Transport Layer Security with TCP

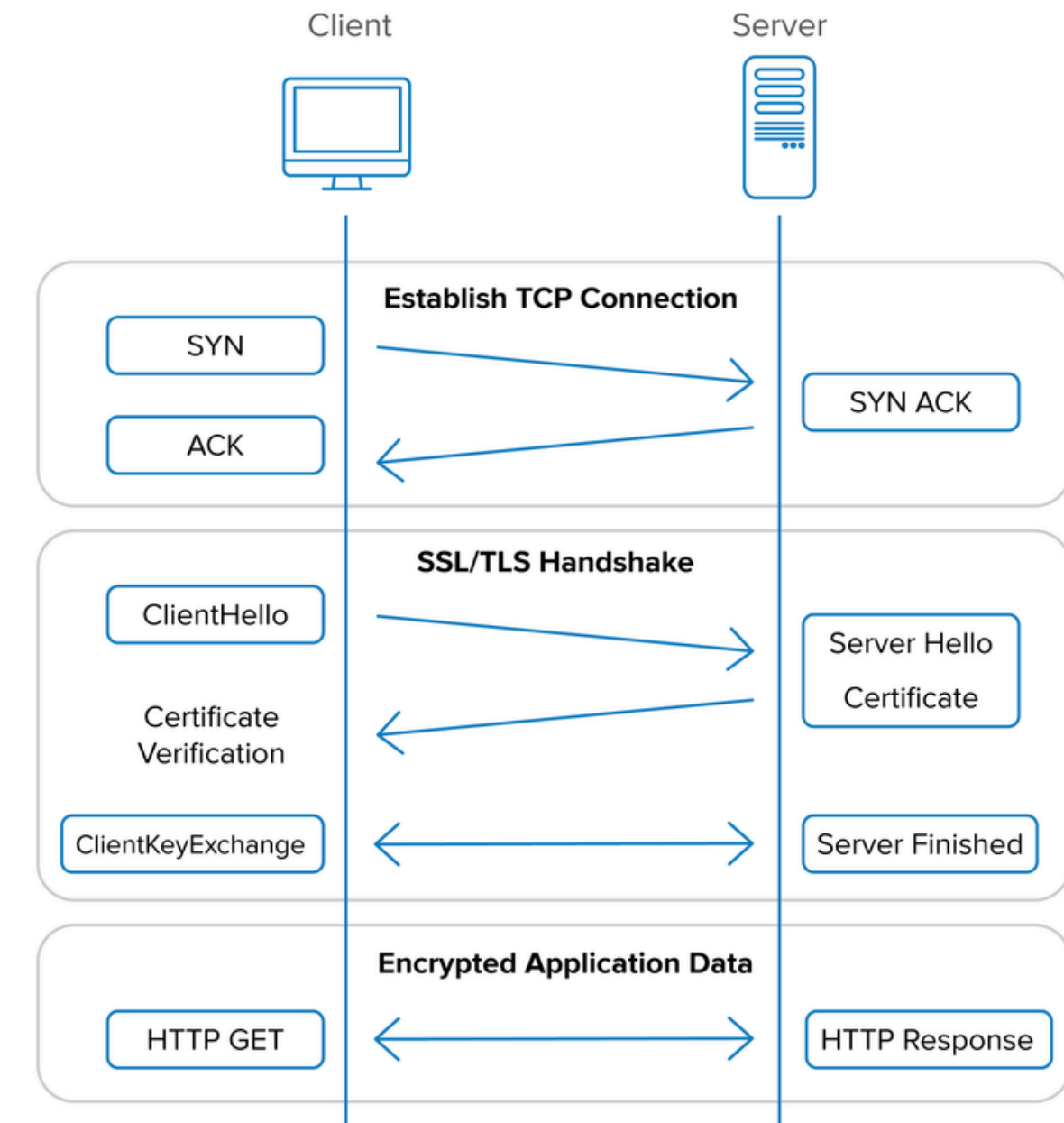
At TCP Layer, how TLS gets added

1. ClientHello

The browser initiates the handshake with:

- Supported TLS versions
- List of supported cipher suites (ways to encrypt)
- Random number (client_random)
- Optional: Server name

This is the browser saying, “Hey, here are the languages I speak. Can we talk securely?”



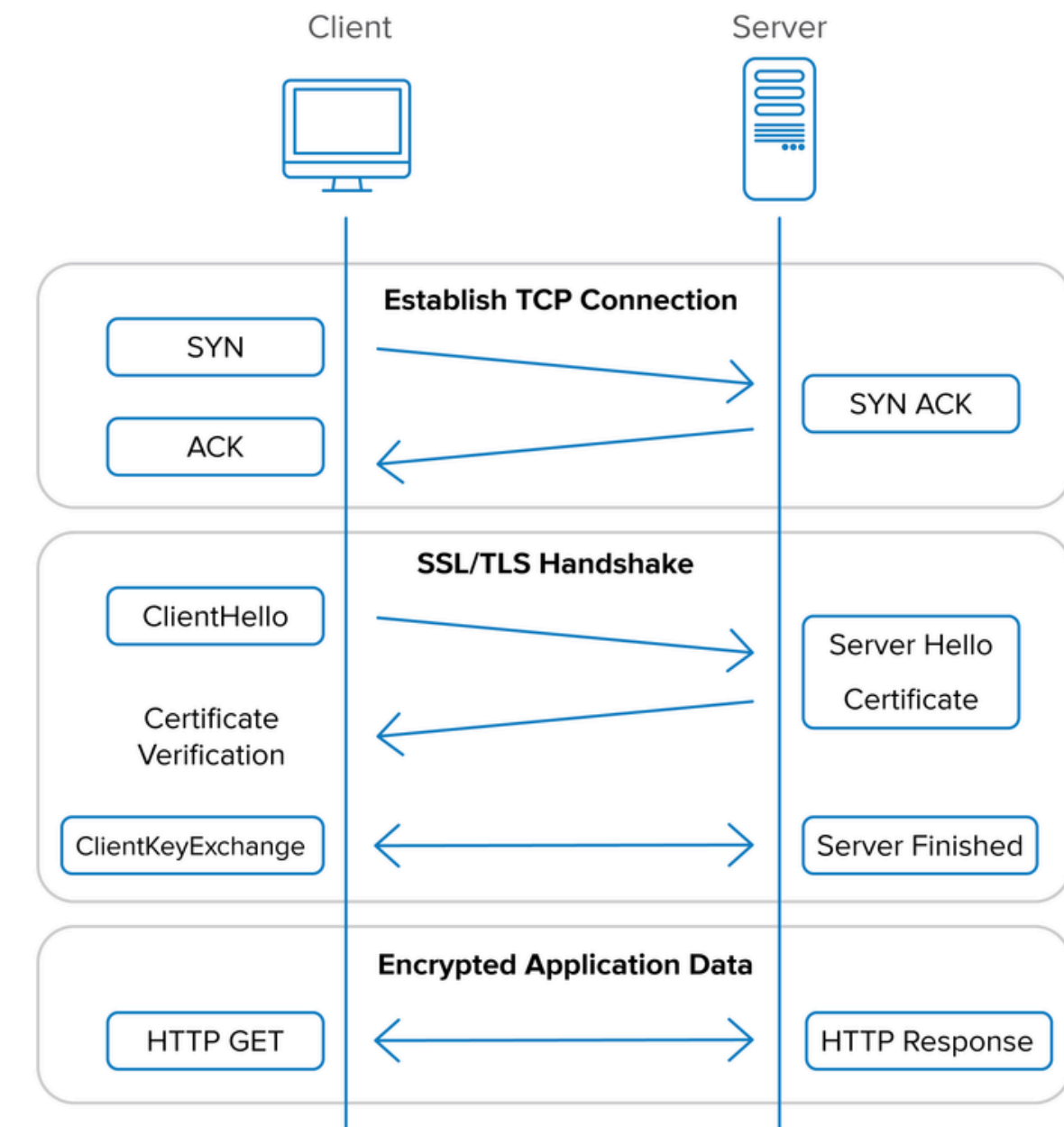
At TCP Layer, how TLS gets added

2. ServerHello

The server responds with:

- Chosen TLS version
- Selected cipher suite
- Its own random number (server_random)
- Digital certificate (proves its identity)

This is the server saying, “Sure, I’ll speak this encryption language. Here’s my ID (certificate) to prove I am who I say I am.”



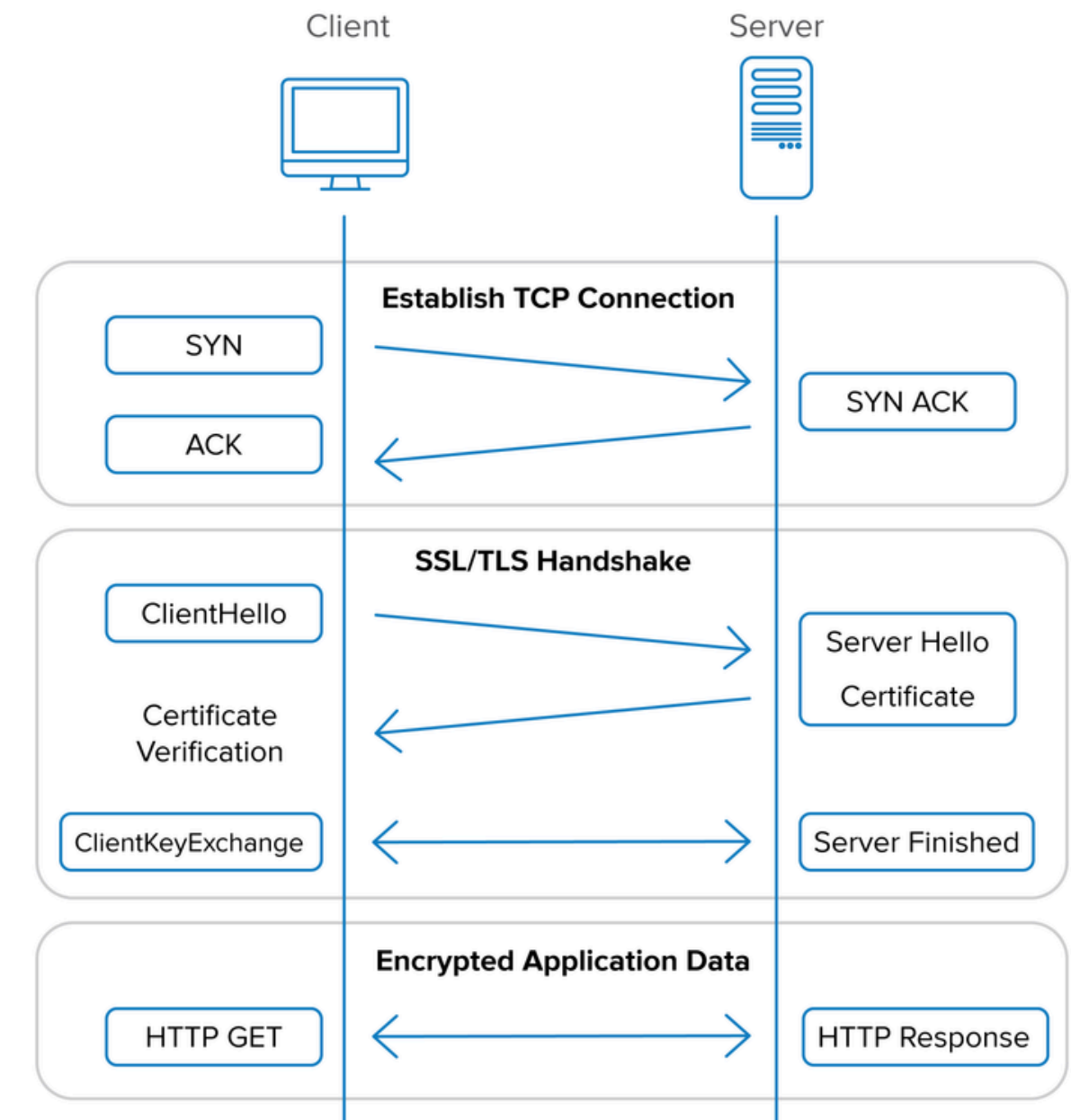
At TCP Layer, how TLS gets added

3. Certificate Verification (on client side)

The client checks if:

- The certificate is valid and trusted (via CA)
- The hostname matches
- It's not expired or revoked

Think of it like checking if a driver's license is real and matches the person.

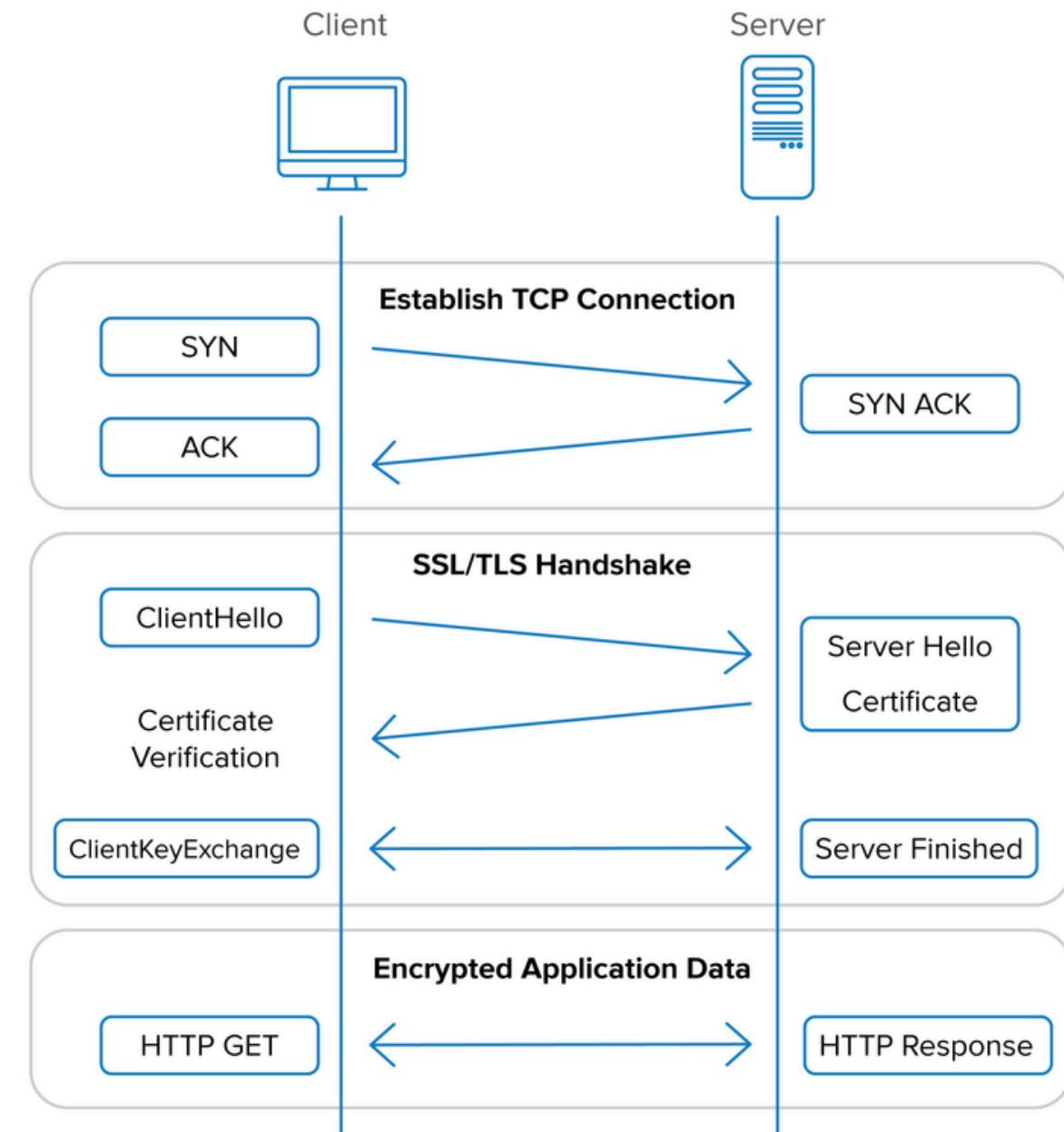


At TCP Layer, how TLS gets added

4. Pre-Master Secret Generation

- Client generates a Pre-Master Secret (a temporary, shared value)
- It encrypts this using the server's public key (from certificate)
- Sends it to the server

Only the server can decrypt this because only it has the private key.



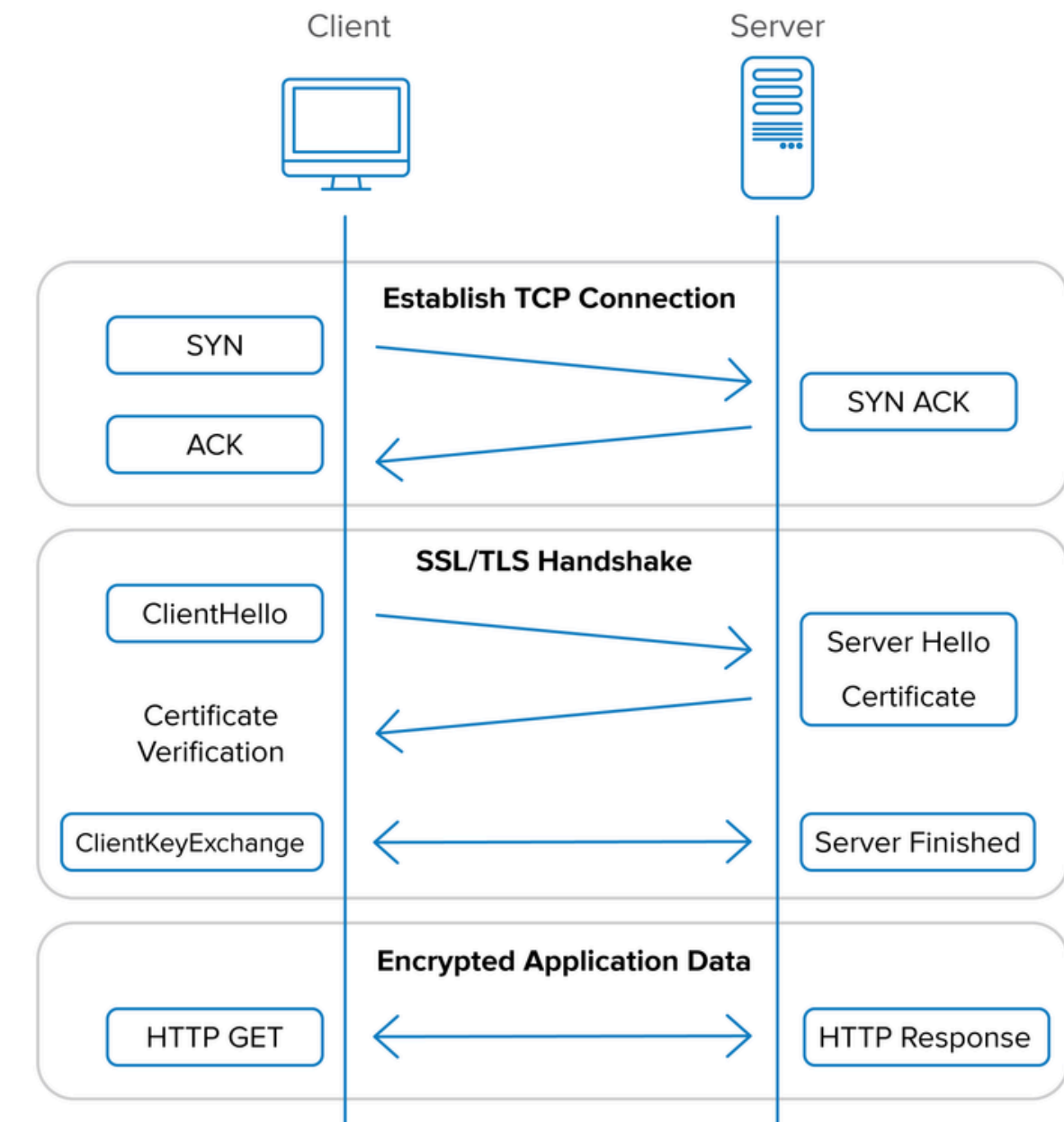
At TCP Layer, how TLS gets added

5. Key Derivation (on both sides)

Using:

- Pre-Master Secret
- client_random
- server_random

Both the client and the server derive the same symmetric session key, which will be used to encrypt communication.

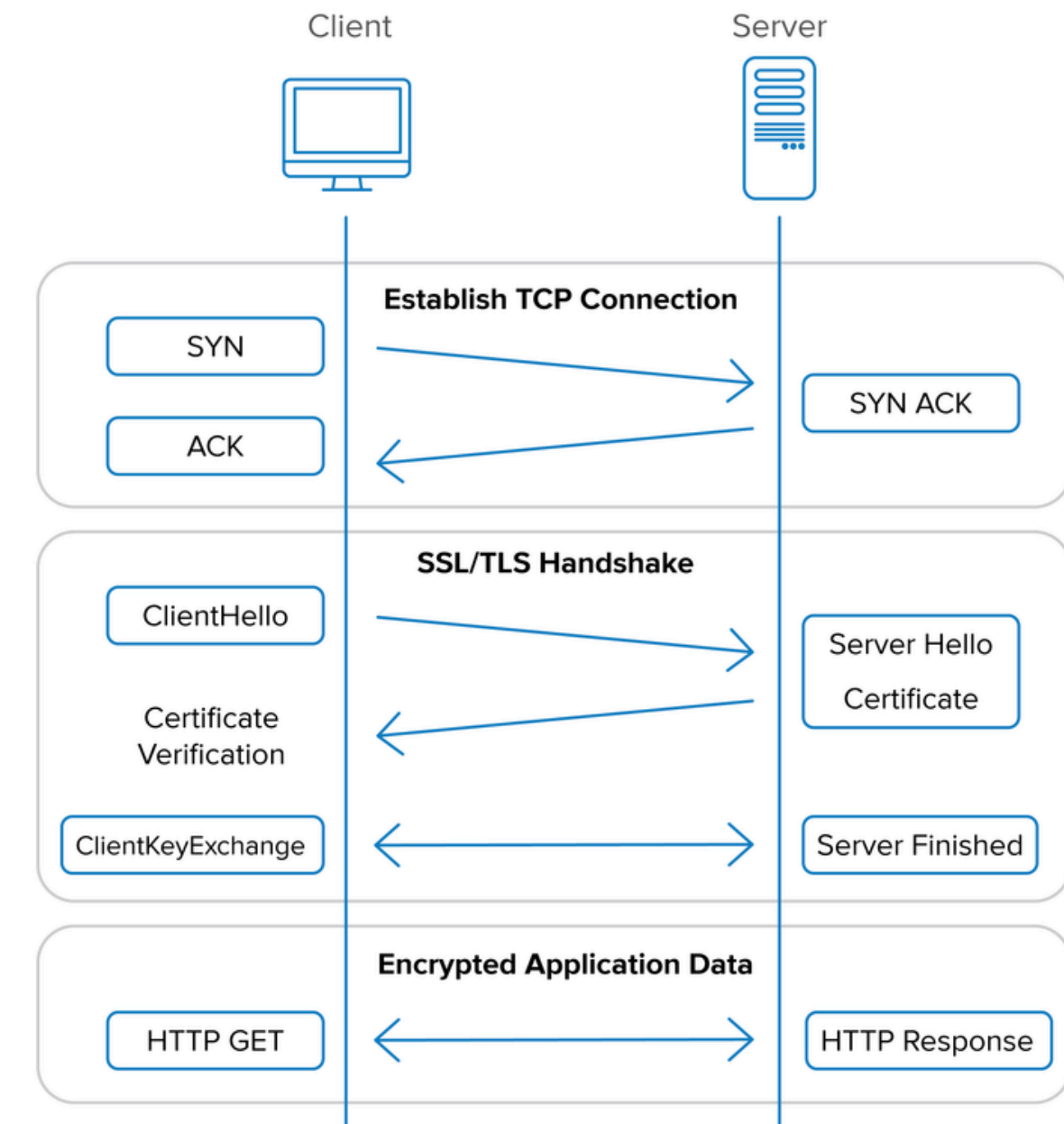


At TCP Layer, how TLS gets added

6. Finished Messages

- Client sends a “Finished” message (encrypted with the new key)
- Server sends its own “Finished” message

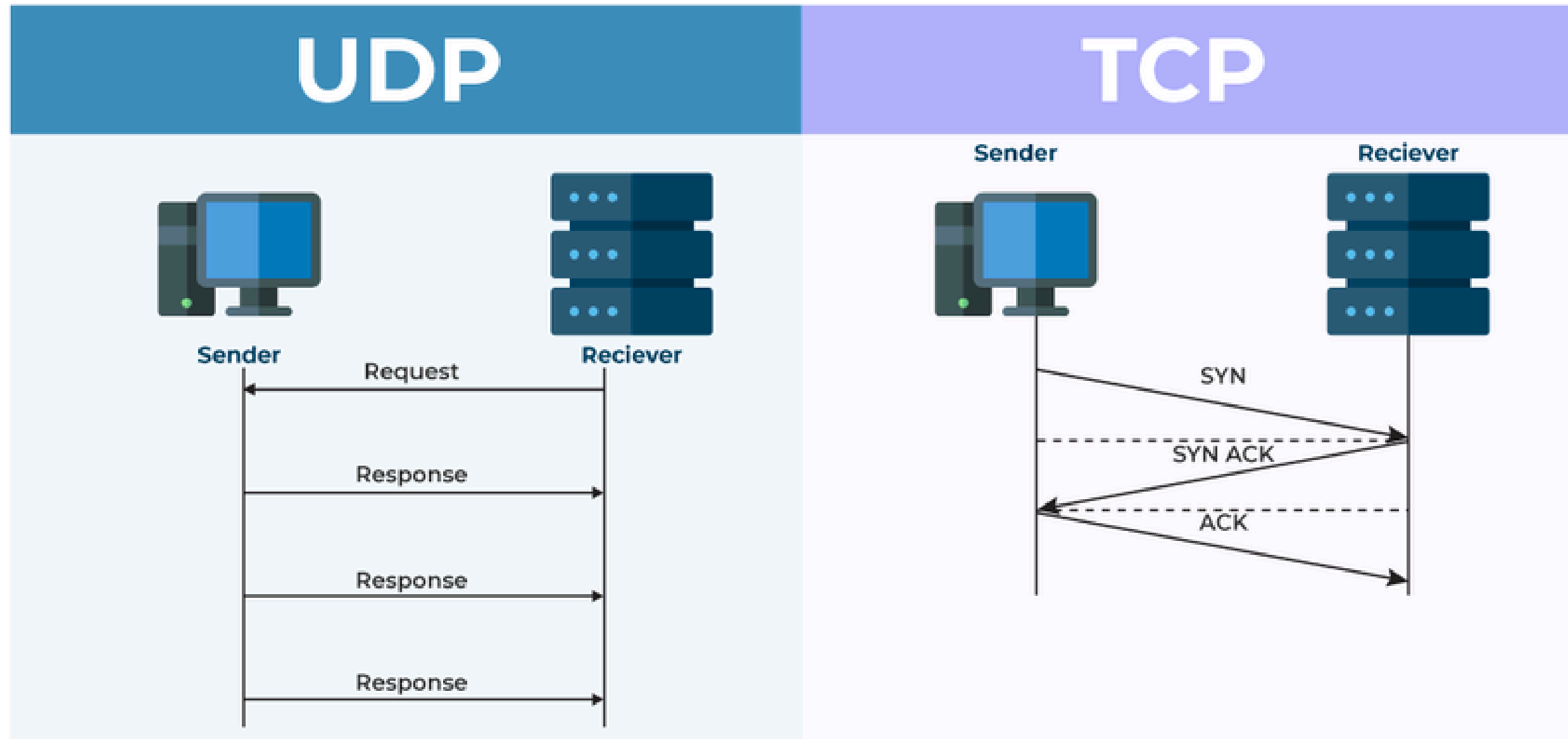
*Now both sides know the connection is secure.
The handshake is complete!*

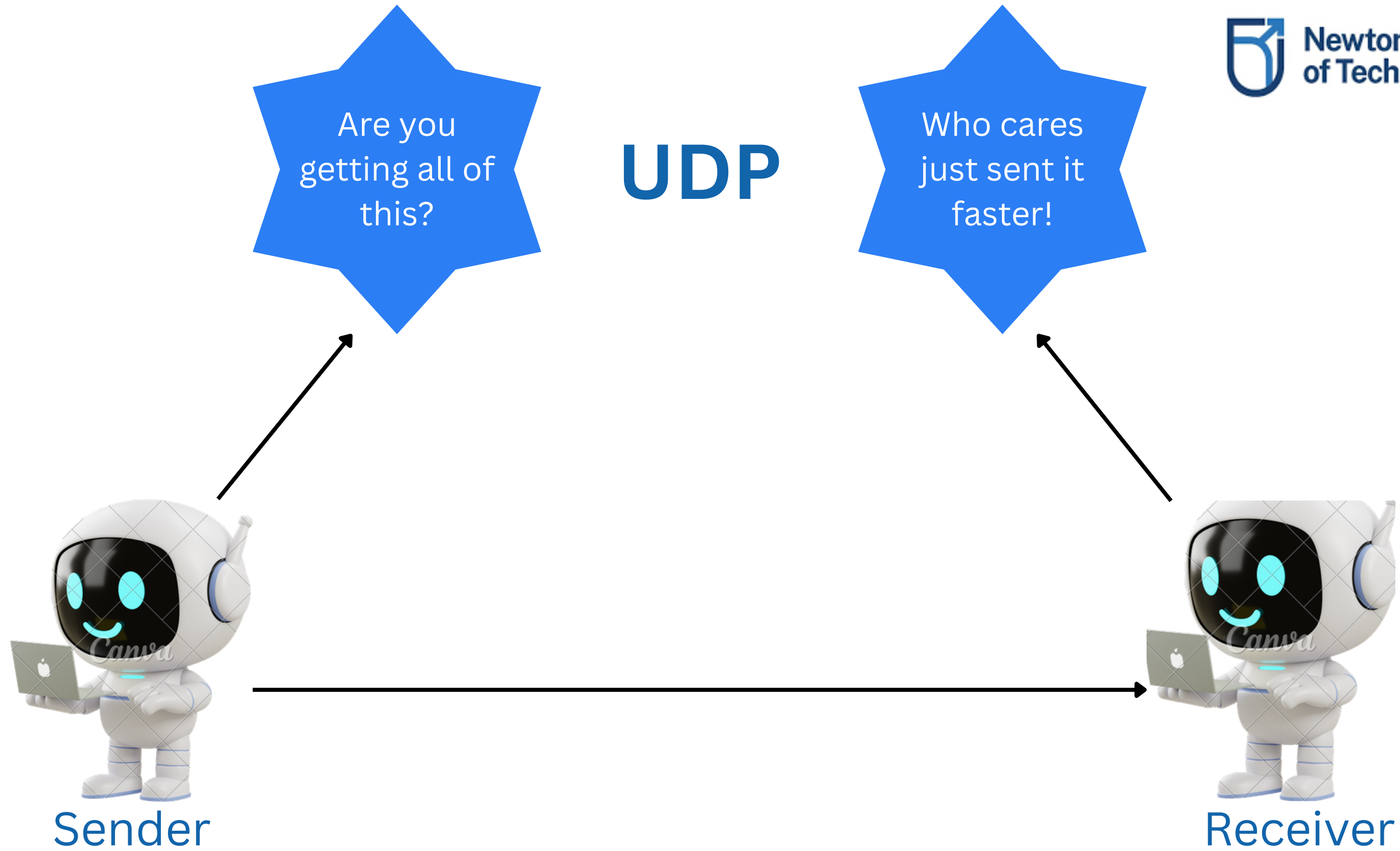


python client TLS exercises

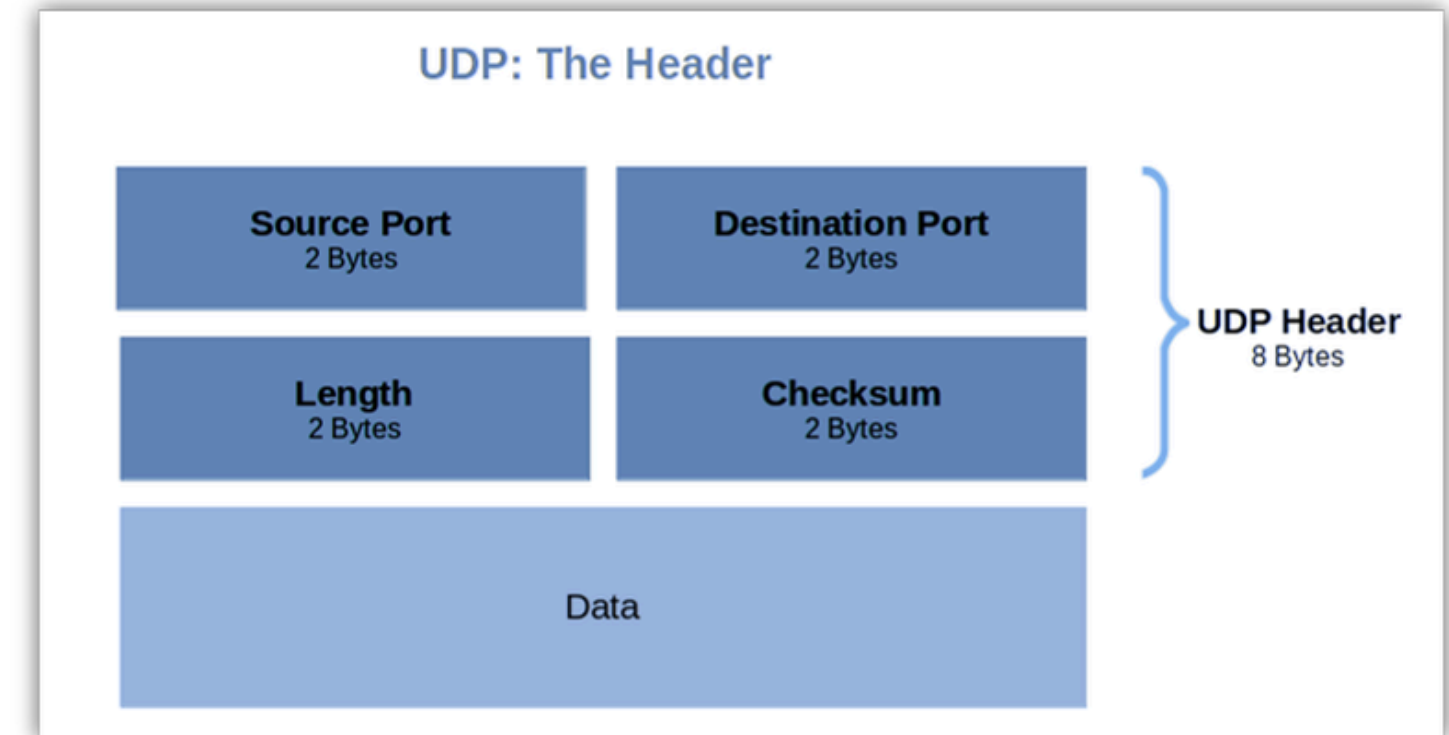
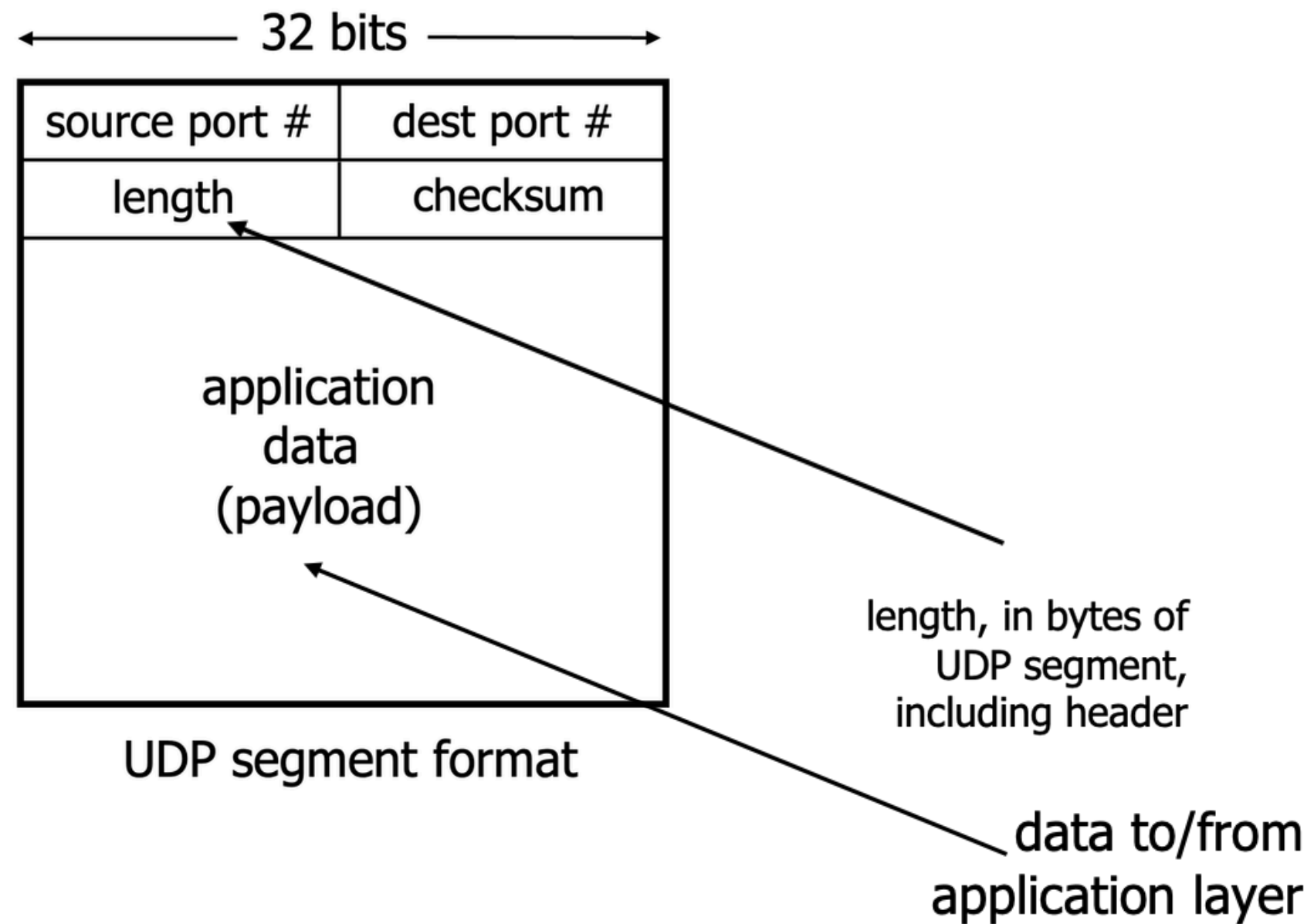
UDP Protocol

Speed vs Reliability – The UDP Trade-Off



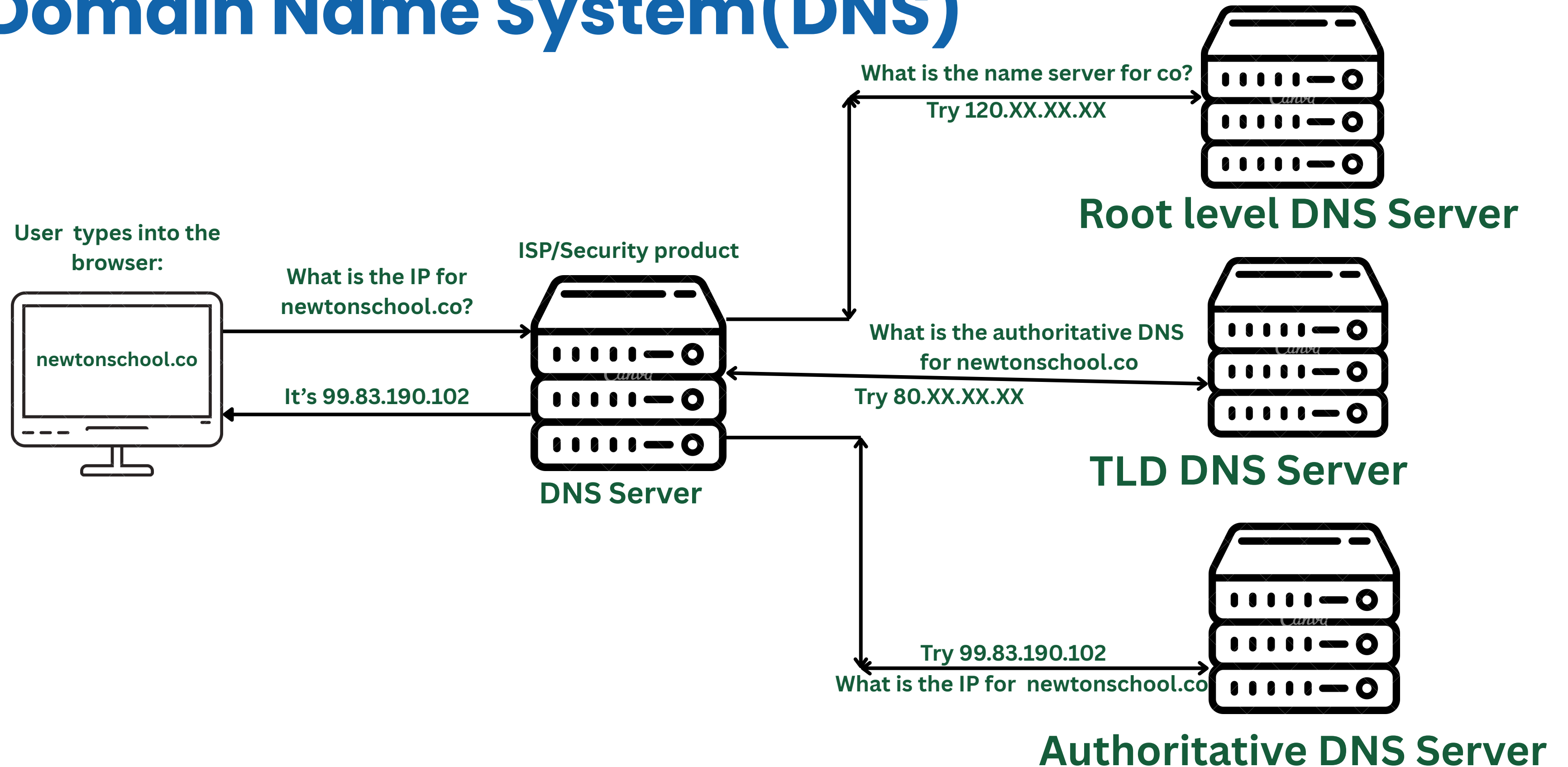


UDP Segment

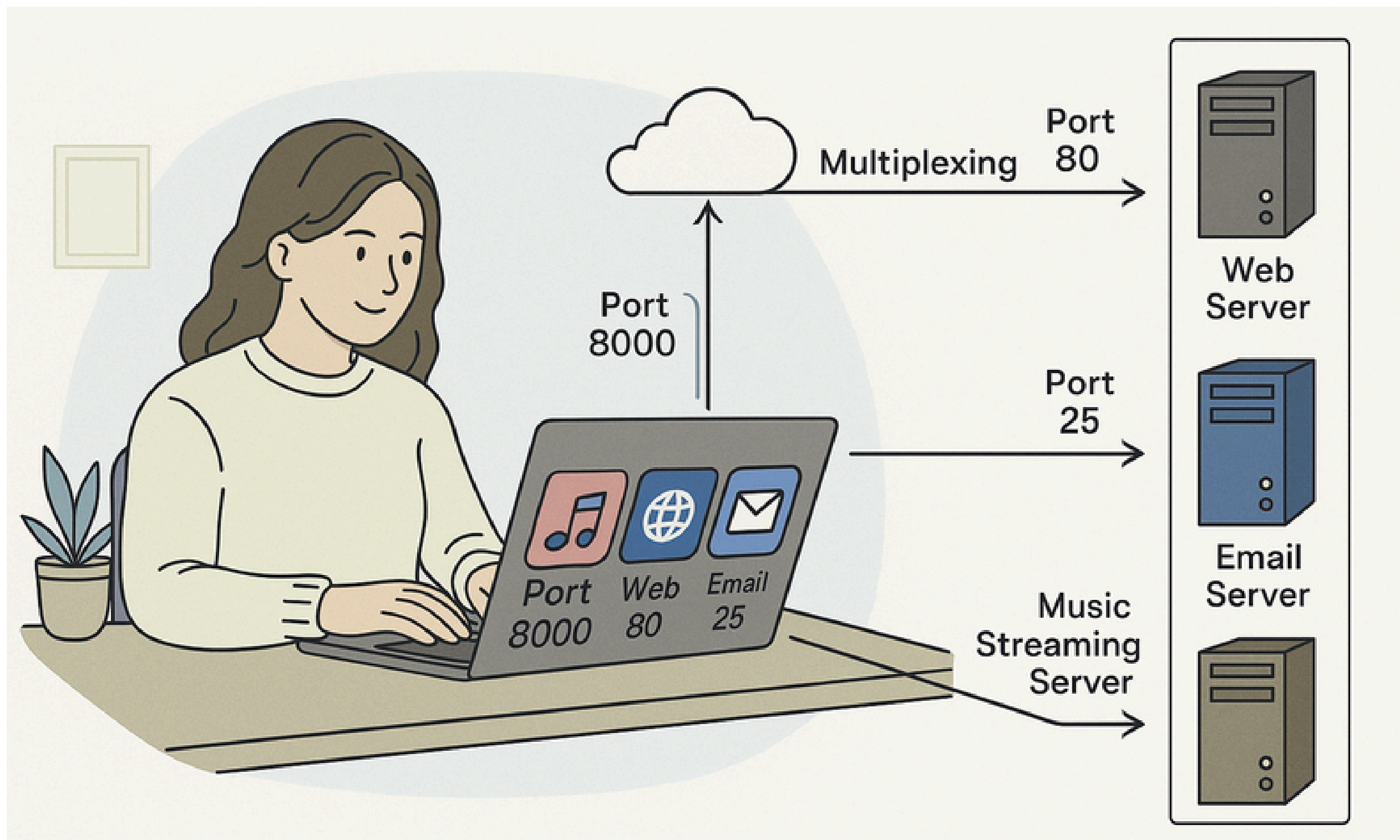


UDP chat using nc

Domain Name System(DNS)

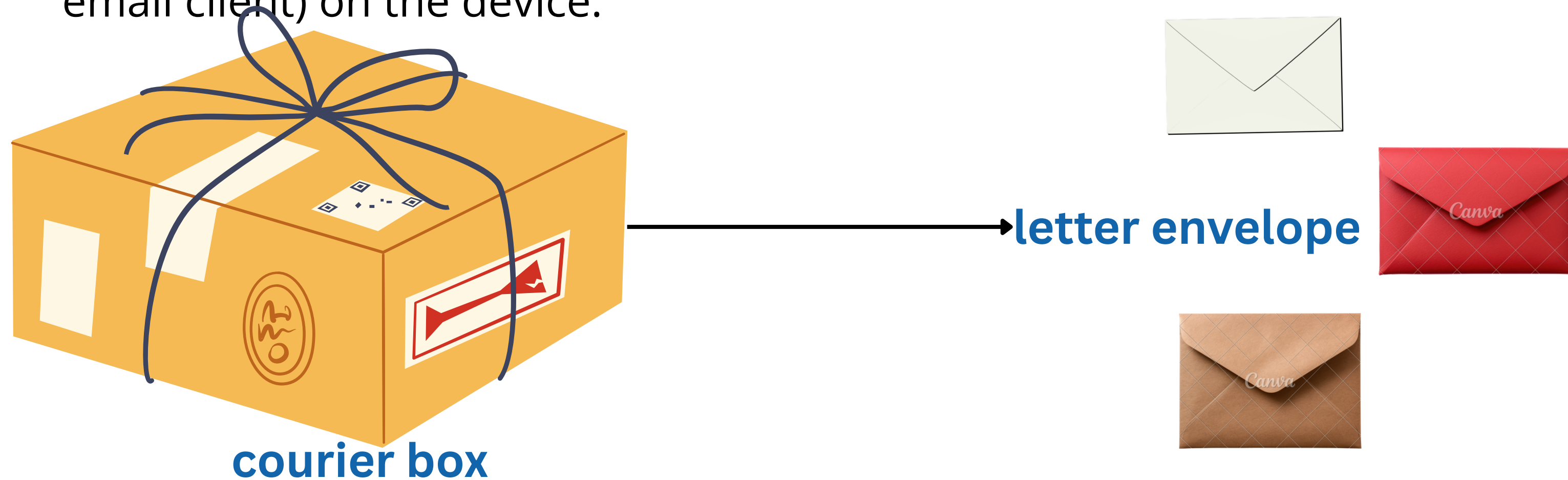


Thank You

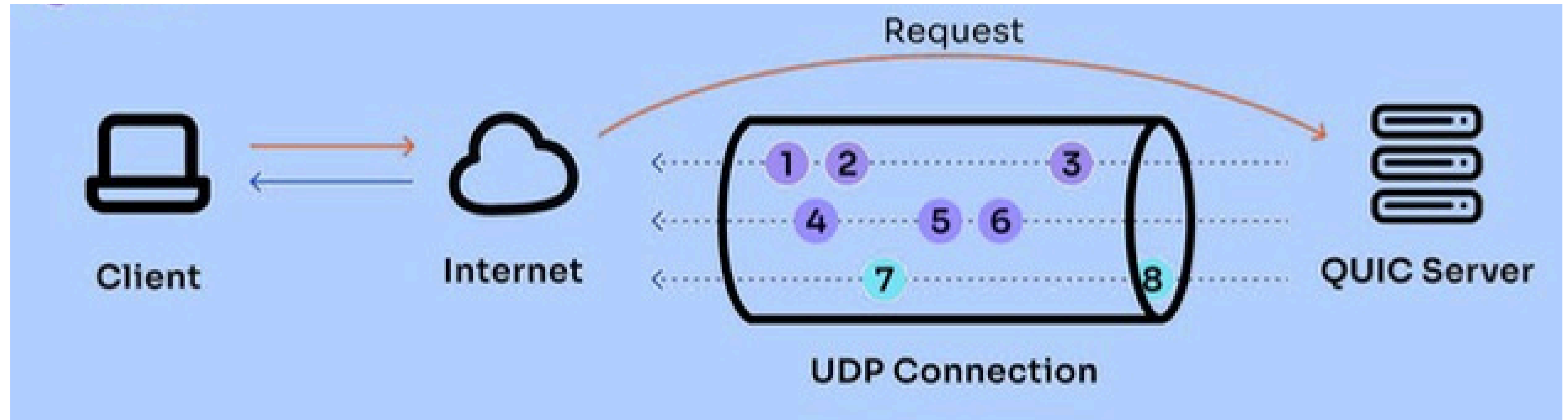


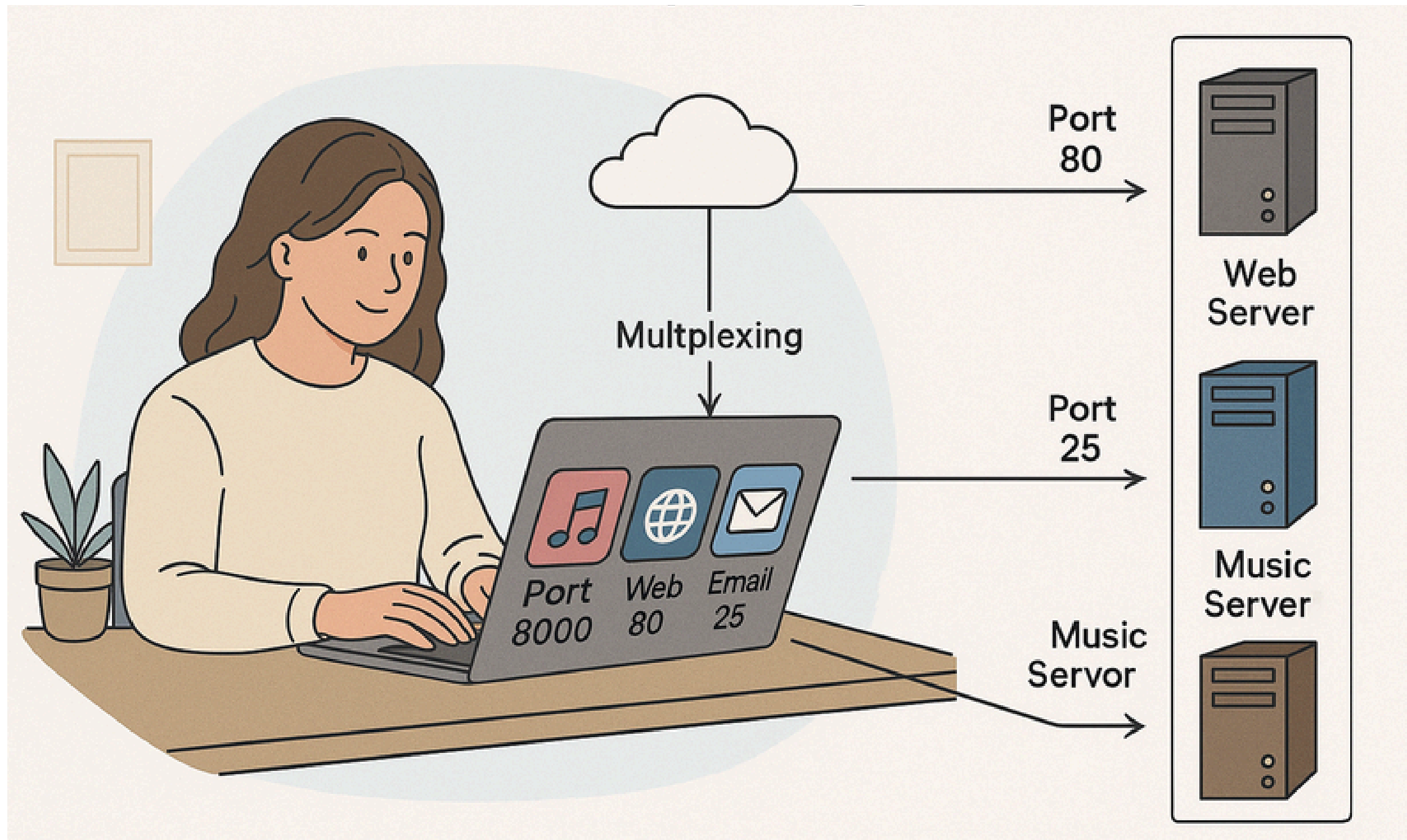
Demultiplexing

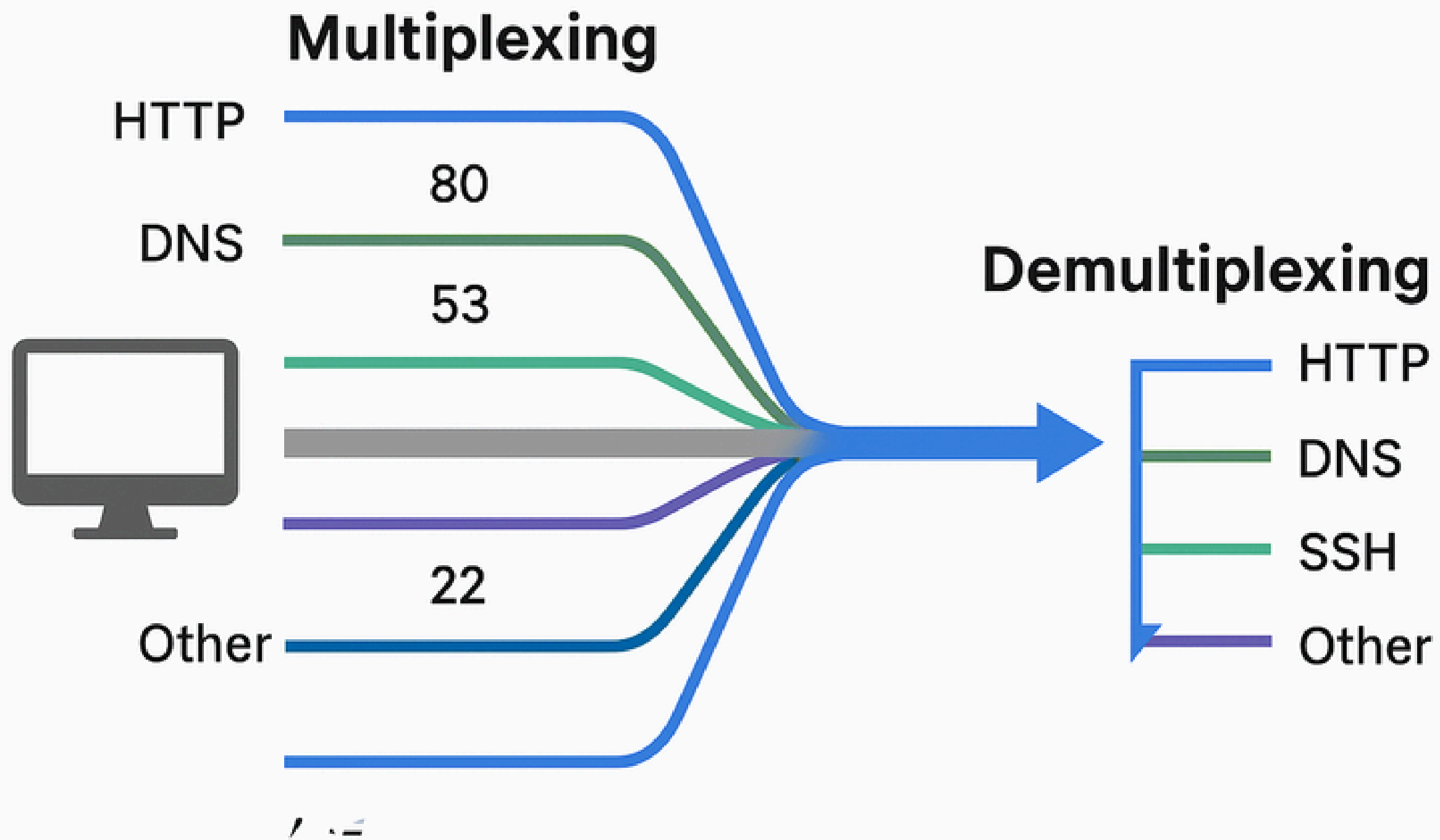
- Separates incoming data at the receiver's transport layer by checking the destination port number on each segment.
- This ensures each segment is delivered to the correct application (e.g., browser, email client) on the device.



Quick UDP Internet Connections(QUIC)

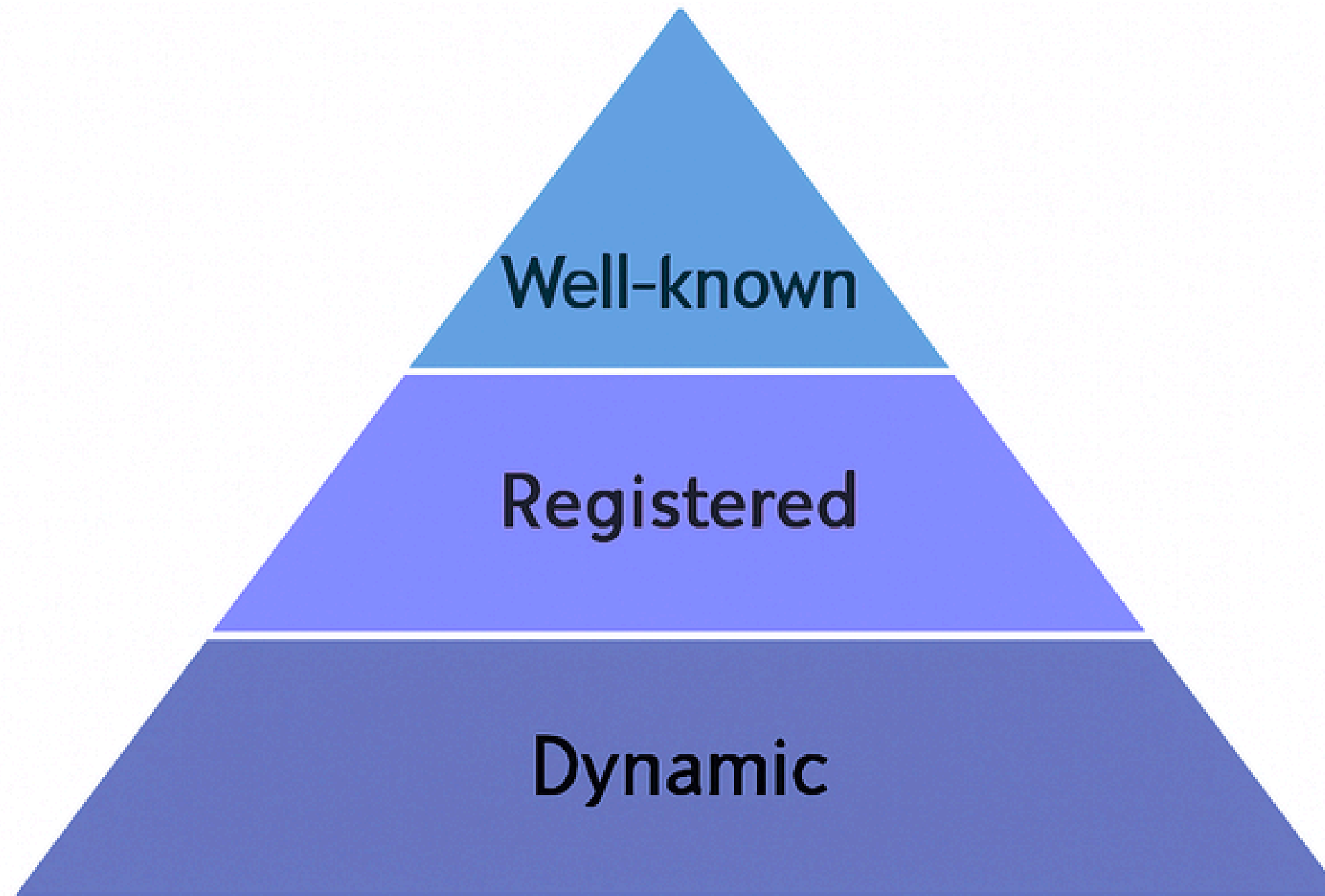






Types of Port Numbers & Common Services

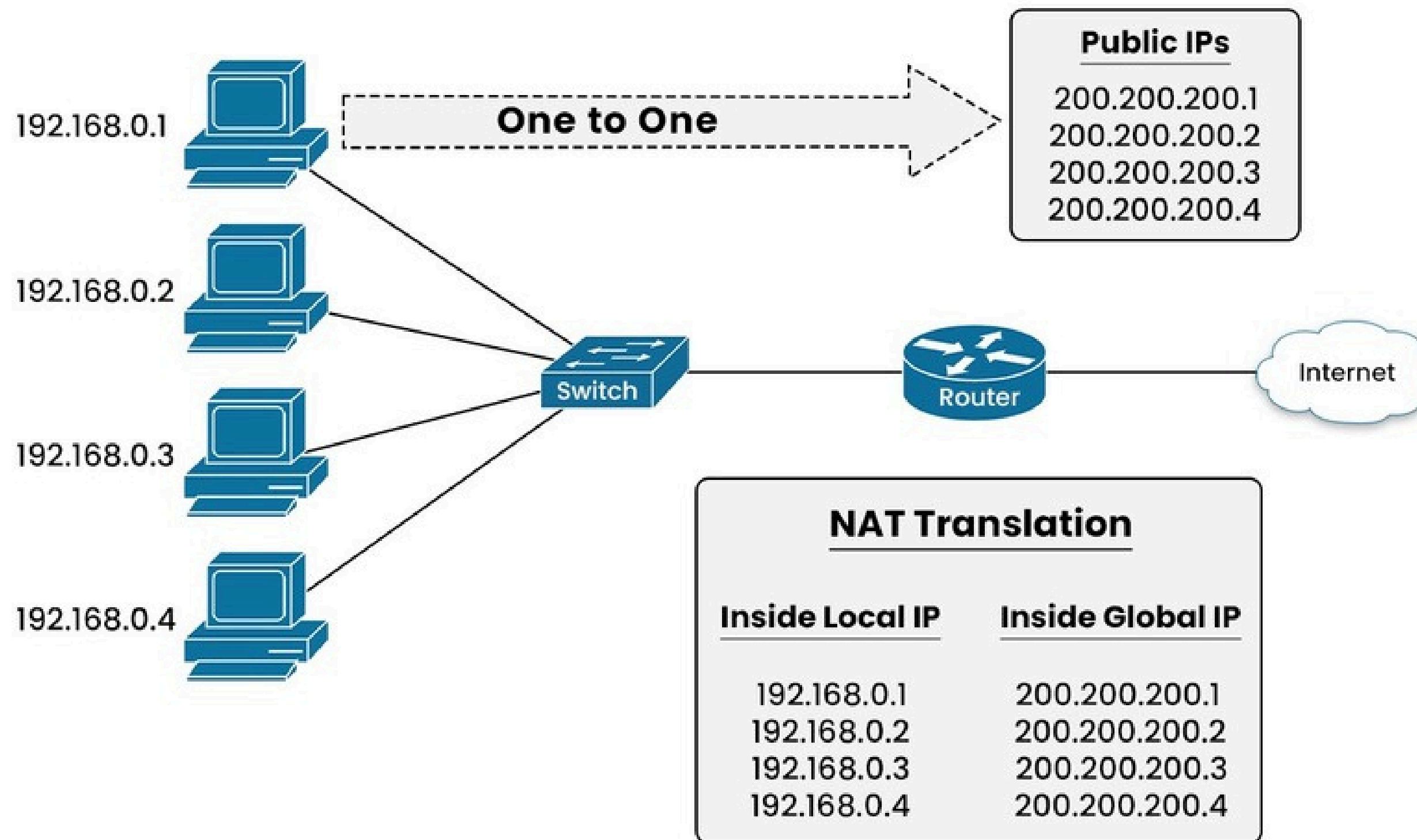
- Port numbers are divided into three main ranges, each serving a different purpose in network communication:
 1. **Well-known Ports (0–1023)**: Reserved for standard services like HTTP(80), HTTPS(443), FTP(21), DNS(53), and SSH(22).
 1. **Registered Ports (1024–49151)**: Used by third-party applications or vendors like MySQL(3306), PostgreSQL(5432), and Docker(2375).
 1. **Dynamic/Private Ports (49152–65535)**: Temporarily assigned by the operating system for client-side connections, such as when a browser initiates a connection.



Types of port numbers

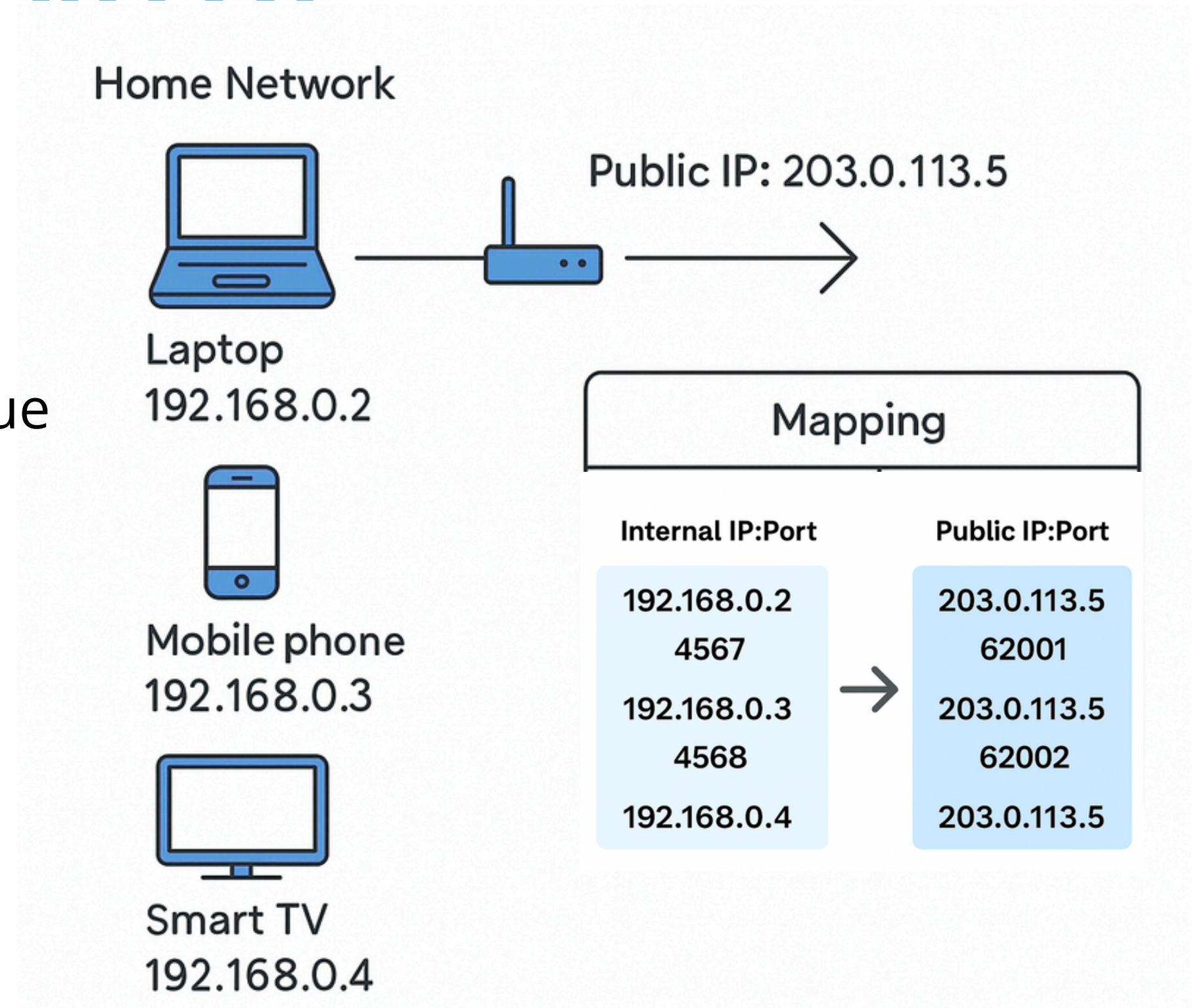
Port Address Translation (PAT)

- Port Address Translation (PAT) is a specific type of NAT (Network Address Translation) that allows multiple devices on a local/private network to share a single public IP address when accessing the internet.
- Instead of assigning a separate public IP to every device, PAT uses unique port numbers to track each connection. This helps conserve public IP addresses and manage many connections efficiently.



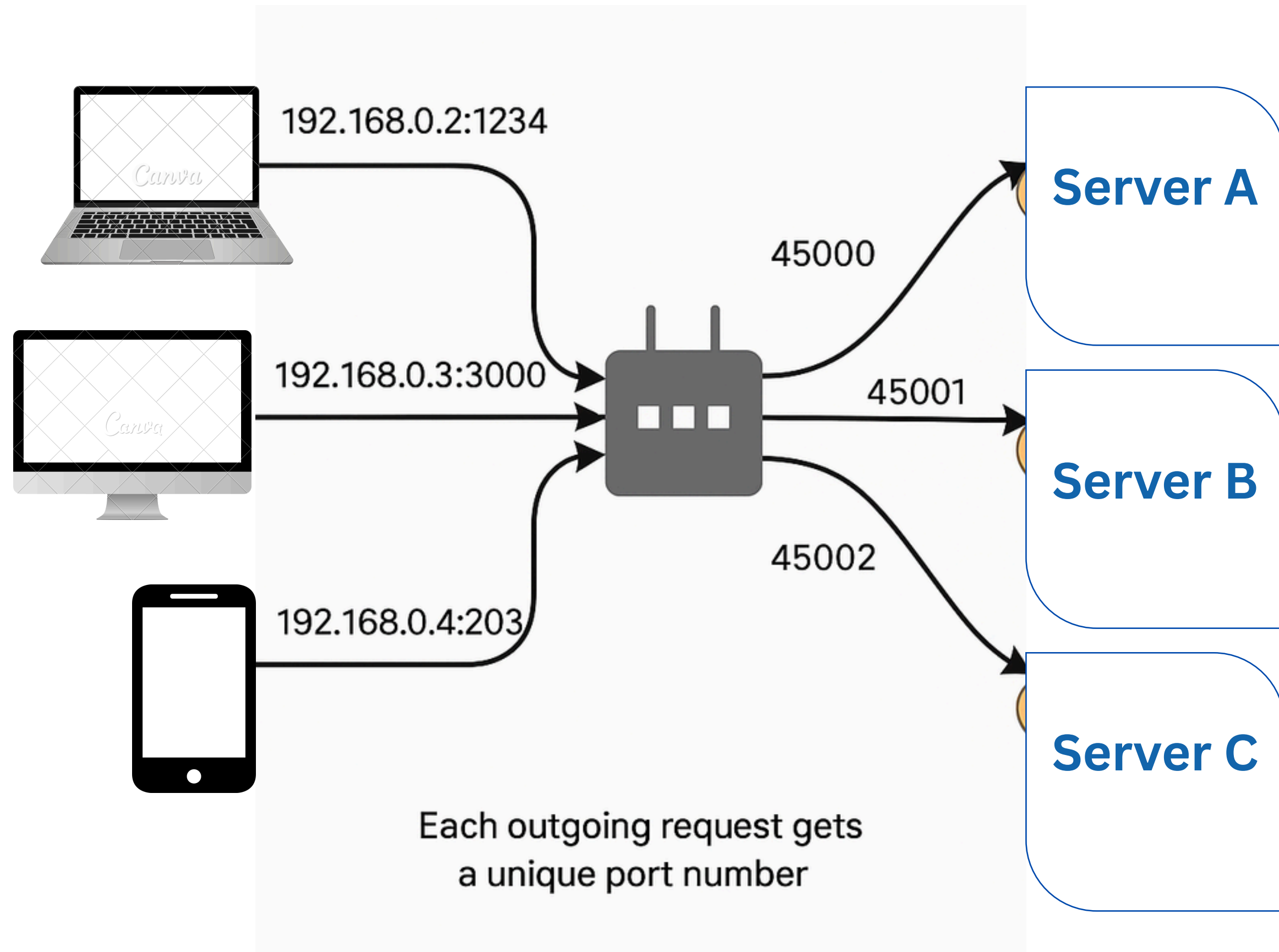
IP & Port Translation in PAT

- PAT doesn't just change the IP address it also rewrites the port numbers.
- Each internal device is given a unique combination of: Public IP address (shared) and Unique port number (per session), This allows many devices to access the internet using one public IP without mix-ups.



How PAT Enables Simultaneous Connections

- Each outgoing request is tagged with a unique port number, even if it comes from the same public IP.
- These unique port numbers act as IDs that help the router track multiple sessions even if they're from the same or different internal devices.
- This allows dozens or even hundreds of devices to connect to different websites or services at the same time, all using a single public IP.



Where UDP Shines – Use Cases

- **Online Gaming:** In online gaming, real-time speed is more important than perfect accuracy, it's better to lose a few data packets than experience delays, as late data is often useless.
- **Simple Request-Response Protocols (SNMP, TFTP):** **SNMP**(Simple Network Management Protocol) uses UDP because it sends small, quick status requests (like checking if a router is online) and doesn't need a connection. If a reply is lost, it simply retries, making it efficient for large networks. **TFTP**(Trivial File Transfer Protocol) uses UDP because it transfers small configuration or boot files in trusted local networks, where speed and simplicity matter more than reliability – avoiding TCP's overhead for faster startup.

Where UDP Shines – Use Cases

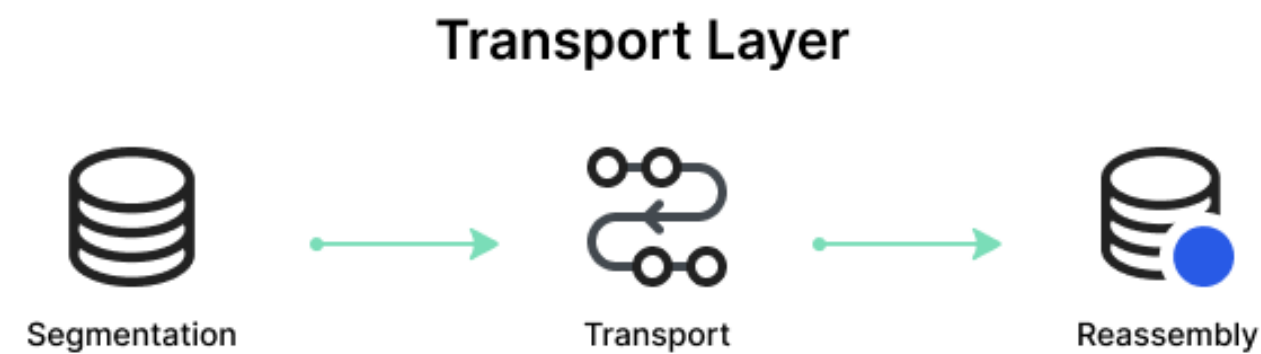
DHCP: DHCP(Dynamic Host Configuration Protocol) uses UDP to quickly assign IP addresses when a device joins a network , it sends one-time requests and responses, relying on the application to handle any needed reliability.

Quick UDP Internet Connections(QUIC)

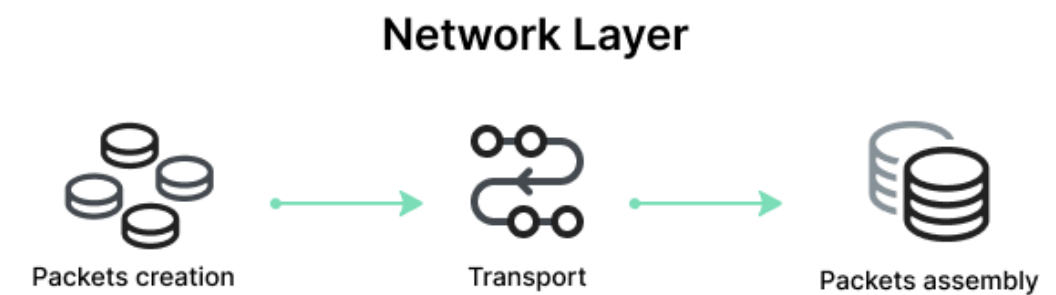
- QUIC is a modern transport protocol built on top of UDP, designed by Google. It's now used in HTTP/3 to make web browsing faster and more secure by combining UDP's speed with TCP-like reliability.
- Unlike TCP, QUIC avoids slow handshakes, efficiently handles packet loss, and offers built-in TLS encryption, faster connection setup, and stream multiplexing making it ideal for platforms like Google, YouTube, and Facebook to deliver faster, more reliable performance.

Join the lecture online on your dashboard

TCP recap



**transport layer: communication
between processes**
segments oriented



**network layer: communication between
hosts**

HTTPS Recap

